

西安电子科技大学

硕士学位论文



基于 UVM 及 VIP 技术的 PCI Express 接口
验证

作者姓名_____徐 航_____

学校导师姓名、职称_____宣荣喜 教授_____

企业导师姓名、职称_____邱钧华 高工_____

申请学位类别_____工程硕士_____

学校代码 10701

分 类 号 TN4

学 号 1411122853

密 级 公开

西安电子科技大学

硕士学位论文

基于 UVM 及 VIP 技术的 PCI Express 接口 验证

作者姓名：徐航

领 域：软件工程

学位类别：工程硕士

学校导师姓名、职称：宣荣喜 教授

企业导师姓名、职称：邱钧华 高工

学 院：微电子学院

提交日期：2017 年 6 月

The Verification of PCI Express Interface based on UVM and VIP Technology

A thesis submitted to
XIDIAN UNIVERSITY
in partial fulfillment of the requirements
for the degree of Master
in Software Engineering

By

Xu Hang

Supervisor: Xuan Rongxi

Title: Professor

Supervisor: Qiu Junhua

Title: Senior Engineer

June 2017

摘要

随着数字集成电路工艺技术的不断提高,数字系统设计的复杂度随之提高,同时对数字验证工作的要求也变得越来越高,采取高效的验证方法对整个芯片的设计完成变得尤为重要。UVM (Universal Verification Methodology) 验证方法学不仅汲取了如 VMM (Verification Methodology Manual)、ERM (e Reusable Methodology) 等其他验证方法的优点,而且继承了 OVM (Open Verification Methodology) 验证方法学的优势,发展成为可以满足大型复杂 SoC (System on Chip) 设计验证需求的新一代验证方法学;Cadence 公司提供了芯片外的功能模型,即 VIP (Verification Intellectual Property) 模型,其中包括了 PCIe (Peripheral Component Interconnect Express)、USB (Universal Serial Bus) 和以太网等。通过结合 VIP 和 UVM 验证方法学设计的验证平台可以很大程度上提高验证效率,加快了整个项目的验证进度。

本文旨在以 UVM 及 VIP 技术为基础设计验证平台,对 PCIe 接口进行功能验证。在分析研究 PCIe 接口以及 UVM 关键技术的基础上,明确了 PCIe 各层的层次结构和数据包格式,同时掌握了 UVM 验证方法学的特点、优势及分层设计验证平台的方法。对 Cadence 公司的 VIP 进行了分析研究,建立了 PCIe VIP 组件,在分析验证对象的结构和功能的基础上,提出验证需求,设计了基于 UVM 及 VIP 技术的可复用验证平台,实现了平台中的各个组件。本文的待测设计 PCIe 接口是 X 型微处理器的外设,该微处理器是基于 PowerPC 架构即精简指令集架构,在进行验证环境配置以及验证激励编写的过程中都需要使用精简指令集中的相关指令,在熟悉整个验证对象的过程中,需要熟练掌握 PowerPC 架构下的精简指令集中指令的应用。对 UVM 和 VIP 技术的应用,缩短了验证平台的设计时间,提高了验证工作的效率;通过定向测试和随机测试对 DUT (design under test) 进行功能验证,其中定向测试针对 PCIe 内部的窗口边界情况进行验证,其他情况由随机测试完成,保证了验证的完备性。最终,功能覆盖率达到 100%,代码覆盖率达到 90% 以上,满足了验证需求。验证结果表明,PCIe 的功能与设计规范中的要求一致,达到了设计要求。

本文设计的验证平台,用于对某项目中 X 型微处理器的外设 PCIe 接口进行验证,整个验证平台能够提供 DUT 所需的所有激励,并且验证平台中的组件复用性高,扩展性强,减少了项目变动给验证平台搭建带来的工作量。

关键词: PCIe, UVM, VIP, 覆盖率

ABSTRACT

With the continuous improvement of digital integrated circuit technology, the complexity of digital system design increases, at the same time the expectation of digital verification has become increasingly high and taking an efficient verification method for the development of the entire chip has become particularly important. UVM (Universal Verification Methodology) can meet the verification needs of large complex SoC (System on Chip) designs, and is a new generation of verification methodology that not only draws on the advantages of other verification methods such as VMM (Verification Methodology Manual), ERM (e Reuse Methodology), but also inherits the advantages of OVM (Open Verification Methodology) verification methodology. Cadence provides off-chip Functional model, that is, VIP (Verification Intellectual Property) model, which includes PCIe (Peripheral Component Interconnect Express), USB (Universal Serial Bus) and Ethernet etc.. Through the combination of VIP and UVM verification method to build a test bench can greatly improve the efficiency of verification.

The purpose of this paper is to design a test bench through UVM and VIP technology to verify the function of PCIe interface. Based on the analysis of the PCIe interface and the key technologies of UVM, the hierarchical structure and packet format of PCIe layers are clarified. At the same time, the structural features of UVM verification method, advantages and the method of building verification test bench are studied. Based on the analysis and research of the VIP, the PCIe VIP component is established. Based on the analysis of the structure and function of the verification object, the verification function points are extracted. Designed a reusable test bench based on UVM and VIP technology, implemented the components of test bench. The PCIe interface is a peripheral of the X-type microprocessor. The microprocessor is based on the PowerPC architecture, which is simplified instruction set architecture. It needs to use the related instruction in simplified instruction set in the process of the verification environment configuration and making stimulus. In the entire process of familiar with verification object, application of the instruction in simplified instruction set which under PowerPC architecture need to be mastered. The application of UVM and VIP technology shortens the time of building test bench and improves the efficiency of the verification. The DUT (design under test) is verified by the directional test (to verify the window boundary condition of the PCIe) and

the random test (to verify other situations), and this ensures the integrity of the verification. Ultimately, the function coverage reached 100% and the code coverage is higher than 90%, met the verification needs. The verification results show that the PCIe function is consistent with the requirements in the design specification and meets the design requirements.

The test bench designed in this paper is to verify the external PCIe interface of a certain X-type microprocessor. The test bench can provide all the stimulus needed by DUT, and the verification components in the test bench are highly reusable and extensible, which can reduce the workload of building the test bench caused by project changes.

Keywords: PCIe, VIP, UVM, coverage

插图索引

图 2.1	PCIe 接口总线的拓扑结构.....	7
图 2.2	交换机逻辑结构图.....	8
图 2.3	PCIe 链路.....	9
图 2.4	PCIe 接口总线的层次结构图.....	10
图 2.5	数据包结构.....	10
图 2.6	TLP 的结构图.....	12
图 2.7	包头 (Header) 结构.....	13
图 2.8	DLLP 结构图.....	15
图 2.9	UVM 验证平台结构图.....	17
图 2.10	验证方法学分层结构.....	19
图 2.11	UVM 树形结构.....	19
图 2.12	UVM 中的 phase.....	20
图 3.1	VIP 组件的生成过程图.....	23
图 3.2	VIP 的一般结构.....	24
图 3.3	The VIP Model in Active Mode.....	26
图 3.4	The VIP Model in Passive Mode.....	26
图 3.5	PureSpec Data Flow.....	28
图 3.6	X 微处理器的框架图.....	31
图 3.7	X 微处理器 PCIe 模块框图.....	32
图 3.8	X 微处理器 PCIe 内部实现结构图.....	33
图 3.9	验证平台基本框架图.....	39
图 3.10	UVM 验证平台框架图.....	41
图 3.11	env 部分程序图.....	42
图 3.12	DUT 与 VIP model 接口连线图.....	43
图 3.13	interface 部分程序图.....	43
图 3.14	agent 模块.....	44
图 3.15	agent 部分程序图.....	44
图 3.16	sequence 部分程序图.....	45
图 3.17	sequencer 部分程序图.....	46
图 3.18	driver 部分程序图.....	47
图 3.19	monitor 部分程序图.....	48

图 3.20	coverage 部分程序图.....	49
图 3.21	memory instance 部分程序图.....	49
图 4.1	X 处理器 IO 地址划分.....	54
图 4.2	验证流程图.....	57
图 4.3	VIP 初始化部分程序图.....	58
图 4.4	DUT 初始化部分程序图.....	58
图 4.5	验证平台.....	59
图 4.6	PCIe 主从模式配置图.....	59
图 4.7	仿真结果图（1）	60
图 4.8	仿真结果图（2）	61
图 4.9	仿真结果图（3）	62
图 4.10	仿真结果图（4）	63
图 4.11	仿真结果图（5）	63
图 4.12	代码覆盖率.....	64
图 4.13	功能覆盖率.....	64

表格索引

表 2.1	事务数据分类.....	11
表 2.2	TLP 数据包的各部分用途.....	12
表 2.3	包头 (Header) 的详细结构.....	13
表 2.3	包头 (Header) 的详细结构 (续)	14
表 2.4	PCIe 接口总线的地址空间和事务类型.....	15
表 2.5	DLLP 数据包类型.....	16
表 3.1	该处理器支持的 PCIe 事务.....	34
表 3.2	PCIe 接口信号.....	35
表 3.3	事务层部分功能点列表.....	37
表 3.3	事务层部分功能点列表 (续)	38
表 3.4	验证组件与 UVM 基本类的对应关系.....	41
表 4.1	PCIe 的端口配置.....	51
表 4.1	PCIe 的端口配置 (续)	52
表 4.1	PCIe 的端口配置 (续)	53
表 4.2	LAWn base address registers.....	55
表 4.3	LAWn attributes registers.....	55

符号对照表

符号	符号名称
Gpbs	G 位每秒
I/O	输入输出端口
GB/s	G 字节每秒

缩略语对照表

缩略语	英文全称	中文对照
PCIe	Peripheral Component Interconnect Express	高速外设互联标准
UVM	Universal Verification Methodology	通用验证方法学
VMM	Verification Methodology Manual	验证方法学手册
OVM	Open Verification Methodology	开放验证方法学
SoC	System on Chip	系统级芯片
DUT	Design Under Test	待测设计
SVA	System Verilog Assertion	断言
VIP	Verification Intellectual Property	验证知识产权
USB	Universal Serial Bus	通用串行总线
SRIO	Serial Rapid I/O	高速互联技术
DDR	Double Data Rate	双倍速率同步动态 随机存储器
PCI	Peripheral Component Interconnect	设备组件互联标准
DMA	Direct Memory Access	直接存储器访问
SVA	System Verilog Assertion	断言
RC	Root Complex	根联合体
EP	Endpoint	端点
TLP	Transaction Layer Packet	事务包
DLLP	Data Link Layer Packe	数据链路层数据包
ECRC	Error Cyclical Redundancy Check	错误循环冗余校验
LCRC	Link Cyclical Redundancy Check	链路循环冗余校验
SOMA	The specification of model architecture	模型说明概况
BFM	Bus Function Model	总线功能模块
CPU	Central Processing Unit	中央处理器
RAM	Random Access Memory	随机存取存储器
ATMU	Address Translation and Mapping Units	地址转换单元
BAR	Base Address Register	基地址寄存器

目录

摘要.....	I
ABSTRACT.....	III
插图索引.....	V
表格索引.....	VII
符号对照表.....	IX
缩略语对照表.....	XI
第一章 绪论	1
1.1 选题背景与意义.....	1
1.2 技术发展现状.....	2
1.3 论文的主要内容和内容安排.....	3
第二章 PCIe 接口总线和 UVM 关键技术分析	5
2.1 PCIe 接口总线的技术特点.....	5
2.2 PCIe 接口总线的拓扑结构.....	7
2.3 PCIe 接口总线的链路.....	9
2.4 PCIe 接口总线的层次结构.....	9
2.4.1 事务层结构.....	10
2.4.2 数据链路层结构.....	15
2.4.3 物理层结构.....	16
2.5 UVM 验证方法学.....	17
2.5.1 UVM 验证平台的基本结构.....	17
2.5.2 UVM 验证方法学的特点.....	18
2.6 本章小结.....	20
第三章 基于 UVM 及 VIP 技术的验证平台设计	23
3.1 VIP 组件.....	23
3.1.1 VIP 组件的生成.....	23
3.1.2 VIP 的结构框架.....	24
3.1.3 VIP 组件的操作模式.....	25
3.1.4 PCIe VIP 数据流.....	27
3.2 验证策略的制定.....	30
3.2.1 验证设计对象.....	30
3.2.2 验证计划的制定.....	35

3.2.3	验证功能点的提取.....	36
3.2.4	验证环境的构建.....	38
3.3	PCIe 验证环境架构.....	38
3.4	验证平台子模块的实现.....	40
3.4.1	顶层 top.....	42
3.4.2	环境类 env.....	42
3.4.3	接口 interface.....	42
3.4.4	代理 agent.....	43
3.4.5	序列 sequence.....	44
3.4.6	激励产生器 sequencer.....	45
3.4.7	驱动器 driver.....	46
3.4.8	监视器 monitor.....	47
3.4.9	覆盖率收集器 coverage.....	48
3.4.10	VIP model.....	49
3.5	本章小结.....	50
第四章	PCIe 验证平台验证实施以及覆盖率分析.....	51
4.1	验证环境配置.....	51
4.1.1	X 微处理器的上电配置.....	51
4.1.2	地址空间配置.....	53
4.2	验证流程.....	56
4.3	验证实施.....	58
4.3.1	事务层验证.....	59
4.3.2	数据链路层验证.....	61
4.3.3	物理层验证.....	62
4.4	覆盖率相关报告.....	64
4.5	验证平台的重用性研究.....	65
4.6	本章小结.....	65
第五章	总结与展望.....	67
5.1	工作总结.....	67
5.2	后续展望.....	67
参考文献		69
致谢		71
作者简介		73

第一章 绪论

1.1 选题背景与意义

随着超大规模集成电路工艺技术的不断进步,数字芯片集成度随之提高,使得数字芯片的验证也变得更加复杂,以往的定向验证已经难以胜任当前超大规模集成电路的验证需要,为了弥补以往验证方法学的不足^[1],通用验证方法学 UVM 应运而生。它继承和融合了开放式验证方法学 OVM 和验证方法学手册 VMM 的优点,应用标准的分层验证框架,实现了验证平台复用性与标准化的问题,提高了验证效率^[1]。

以往的定向验证,以直接测试(directed tests)的方式,向 DUT 提供特定验证场景的激励,等到仿真结束后,通过手工或者自测的方式对结果进行检查。其次,在直接测试的验证平台中,也可以通过产生随机数的方法进行验证,而不是通过将预先设定的值直接写进相应的数据单元中^[2]。对于小设计而言,适合使用直接测试方法,而对于拥有上千个测试用例的芯片规模而言,直接测试根本无法满足验证要求。因此需要新的验证方法学去适应大规模芯片的验证,从而尽量缩短验证所消耗的时间。UVM 是基于 SystemVerilog 开发的一套开源类库,它采取最优的验证结构去完成整个 DUT 的测试,并且有效的结合了带约束随机激励产生、自测试平台等特点^[3]。这些特点的使用可以使验证工程师依据自己的需求去设计搭建验证平台。当需要针对某个 DUT 进行验证平台的设计搭建时,整个平台最重要的是验证平台的质量,采取正确的验证策略,应用产生带有约束随机激励的方法,以自动化方式产生测试情节,这种方法不但可以保证验证平台的质量,并且可以缩短验证周期。

与此同时,随着 CPU 的高速发展,导致总线的数据传输速度成为约束计算机传输速率的关键,使计算机的性能被限定。因此接口总线的发展至关重要,PCIe 已经成为被业界所认可的第三代总线标准,它可以实现之前 PCI 所面临的各种问题。PCI 的数据传送速度仅有 133MB/s,满足不了当前复杂数据传送要求,其次它不能通过主频的提高或者电压的降低而灵活的调整传输速率^[4]。与之不同的是 PCIe 采用点对点的串行互联技术,双通道技术,不仅扩展灵活,而且支持热插拔和热交换。仅仅 PCIe x4 模式就可以达到 2GB/s。相比较而言,PCIe 技术将成为主要的总线标准,对其研究具有重要的意义。

本文重点讲述基于 UVM 和 VIP 技术针对 PCIe 接口进行验证平台的设计搭建,同时对 PCIe 接口进行完整的功能验证,通过 UVM 层次化的验证平台以及 VIP 组件的应用,使得针对 PCIe 接口的验证平台搭建更加高效,验证结果更加准确。

1.2 技术发展现状

随着芯片制造工艺技术向更小尺寸的进步（65nm、45nm、28nm、20nm），超大规模多核 SoC 的出现，专用集成电路设计发展到了一个更高阶段，导致国内外的 IC 公司对验证方法的研究变得更加重视。2000 年，e 语言在验证平台中的应用，标志着 IC 验证平台的搭建开始走向面向对象语言使用，2006 年，VMM 由新思科(Synopsys) 公司推出，并得到了许多 IC 公司的采用，标志着验证语言走向了 SystemVerilog 语言，2008 年，验证构架具有可复用的 OVM 由 Cadence 公司和 Mentor 公司联合推出，并且得到了广泛应用。随着数字 IC 设计的复杂程度在不断增高导致业界急切的需求在功能验证工作方面有新的技术和方法学^[5]，在 2011 年 2 月，新一代验证方法学 UVM 正式发布，它是基于 SystemVerilog 语言的新的验证方法学，它继承了 OVM 和 VMM 验证方法的优点，克服了各自的缺点，同 OVM 相比，UVM 的功能更加强大，与 VMM 相比，UVM 的架构层次清晰，源码可读性好等。

对于验证工程师来讲，当前有许多方法可以编写验证平台，设计验证环境。其中包含了通过验证工程师手动编写代码进行单独的直接测试，还包含了采用带约束随机激励的产生，利用自动产生新测试用例方法的先进验证平台^[6]。目前最流行的验证技术是利用功能覆盖率统计进一步加强自动验证效率。业界普遍使用随机验证方法对 DUT 进行验证，只有极少数的小型模块才会采用定向验证方法。而对于随机验证方法来说，当前国内外应用最广泛的是基于 UVM 验证方法学，使用断言的验证方法和使用 VIP（验证 IP）组件的验证方法进行验证平台的设计搭建。

断言（System Verilog Assertion SVA）是设计属性的描述。对于断言而言，它是一种描述性语言，可以检查设计意图，诊断设计缺陷，它不仅对时序相关状况有很好描述，而且自身精确易于维护，可以内嵌函数去验证特殊的情景，并且可以以自动化的方式收集功能覆盖率^[7]。在验证实施的进程中，对于一个被测试对象来说，若是被测试的属性跟预期结果不同，则这个断言失败；若是在模拟过程中某个禁止的属性被呈现出来，则这个断言失败。在当前，断言已经被用于多类方法学中，同时也被许多公司采用，可以缩短验证周期。

VIP 其实本身就是一种 IP，但是它仅仅用来验证设计，它的本质是一系列验证代码的集合，是针对某个具体设计或 IP 开发的，其中包含了许多实现不同功能的验证部件，其次包含了产生激励、创建响应和检查总线接口协议等功能，这些功能可以提高 SoC 验证效率；应用 UVM 验证方法学以及 VIP 技术，通过使用分层的约束随机验证环境，可节省一半以上的测试平台开发时间^[8]。VIP 作为一种可重用的验证组件，广泛的应用在总线接口协议中，从随机验证方法学被广泛应用以来，各类接口总线的 VIP 依次被开发和应用，用来提升验证组件的复用性，VIP 组件可以重复应用于多个 IP 验

证环境中，利用验证组件可以很大程度上减少验证环境开发进程，提高工作效率。对于本课题基于 UVM 及 VIP 组件的 PCIe 的接口验证采用了 Cadence 公司的 VIP 组件，进行验证环境的搭建，大大提高了工作效率，同时为以后项目中的验证工作积攒了一定的经验。

1.3 论文的主要内容和内容安排

本文的研究内容是通过搭建 UVM 平台以及在 UVM 平台中使用 VIP 组件对第三代总线 PCIe 接口总线进行验证，其中论文主要研究分析了 PCIe 主要内容和关键技术，重点分析了 VIP 组成部件的功能以及在 UVM 环境上的使用。通过设计的 UVM 验证平台，产生带约束的随机化测试向量对 PCIe 接口协议进行功能验证，完成验证功能点的测试，并且使验证功能覆盖率达到 100%，代码覆盖率达到了 90% 以上。

本论文的重点是进行基于 UVM 及 VIP 技术进行验证平台的设计搭建，对 PCIe 接口协议进行功能验证，论文的结构安排如下：

第一章主要论述了本次课题的选题背景和意义，表明了研究该课题的必要性，其次介绍了验证方法的技术发展现状，以及本论文的主要内容和组织架构。

第二章主要针对 PCIe 接口总线协议和 UVM 验证方法学的关键技术进行了研究分析，其中主要包括 PCIe 各层次的主要内容和关键技术，其次描述了 UVM 验证方法学的基本结构特点等。

第三章对 Cadence 公司 VIP 的生成、结构框架、操作模式以及 PCIe VIP 的数据流进行了详细的分析研究，通过验证策略的制定设计了基于 UVM 及 VIP 技术的验证平台架构，并且对验证环境中的组件实现进行了阐述。

第四章主要是通过验证环境的配置、验证流程以及验证实施，对 X 型微处理器的外设 PCIe 接口进行功能验证，并且通过测试用例的举例，详细说明验证功能的正确完成。

第五章主要是对论文的研究做了整体的工作总结和后续的展望。

第二章 PCIe 接口总线和 UVM 关键技术分析

随着 CPU 的飞速发展, CPU 的速度已经远远超越了总线速度, 而作为上一代总线技术的 PCI 总线技术, 其总线传输速度已经满足不了其他设备的需求, 导致了整个计算机系统的性能受到了很大程度上的限制^[9]。因此第三代 I/O 总线技术即 PCIe 接口总线技术顺势而生。作为第三代高性能总线的 PCIe 总线, 其作用是用于计算机和通信平台等应用的外围 I/O 互联。PCI-SIG (PCI 特别兴趣小组) 在 2002 年 7 月 23 日发布了 PCIe1.0 版本规范, 发布之后就快速的得到了行业内的肯定, 并且被业界一致公认为 PCIe 总线为下一代总线标准。随着时间的推移, PCIe 接口总线技术也在不断的提高和发展, 在 2005 年 3 月 PCI-SIG 推出了 PCIe1.1 版本规范, 2007 年 PCIe2.0 版本正式推出, 是当前运用最多, 最成熟的版本之一, 2012 年 PCIe3.0 版本正式诞生, 3.0 版本不仅增加了发送器/接收器的均衡、数据时钟恢复等新功能并且对 PLL 进行改良, 提高了系统的性能。

2.1 PCIe 接口总线的技术特点

相比较于上一代 PCI 接口总线技术, 第三代总线技术 PCIe 接口总线技术具有显著的技术优势和技术特点, 它解决了 PCI 总线技术所面对的各类问题, 可以将各种外设与系统内的 PCIe 接口总线互联^[10], 直接进行数据交互, 由于 PCIe 接口总线技术有鲜明的技术特点, 它也被业界迅速的认可, 被公认为第三代总线技术, 下面详细说明 PCIe 接口总线具有哪些技术:

1. 支持设备之间的点对点串联技术

相比较于 PCI 接口总线全部设备共享同一条总线资源, PCIe 接口总线采用点对点串联技术^[11], 对于 PCIe 设备来说, 它不用跟其他设备分享资源, 而独自占用通道带宽, 这样就可以保障各设备的宽带资源^[10], 从而很大幅度的提高了数据的传输速率。

2. 支持全双工, 高带宽

PCIe 接口总线采用全双工传输模式进行数据传输, 数据发送端和接收端总线分别各自独立工作, 这样很大程度的提高了数据的传输效率。相比较于 PCIe 1.0 版本中信道单方向的传输速度最高为 2.5Gpbs, PCIe 接口总线 2.0 版本信道单方向的传输速率高达 5Gpbs^[12], 同时物理层每条链路包括了 x1、x2、x4、x8、x12、x16 等多条信道可供选择, 使得接口的扩展传输更加容易。

3. 具备扩展灵活的能力

同 PCI 接口总线技术相比, PCIe 接口总线的另一个技术特点, PCIe 接口总线能

够延长到系统外部,采取专用的线缆,能将系统外部的全部设备与系统内的 PCIe 接口总线直接相连,好处在于许可开发商开拓出能与主系统分离的高性能的控制产品,而没有必要去改用 USB 等其他接口技术。

4.支持(QoS)服务质量链接配置

在 PCIe 中,服务质量可以用于预测带宽和预测在 PCIe 结构中不同事务流通过的延迟时间。

5. 多个虚拟通道(TC)在所有物理链接中被提供

针对 PCIe 接口总线设计的配置软件会为每一条链路建立虚拟通道,并且多个虚拟通道被支持在每一条链路上。从理论上讲,由于每条物理通道都容许 8 条虚拟通道进行单独的通信控制,所以任意一个通信的数据包都分别定义了不同的虚拟通道优先级,这样就可以保证获得充足带宽在实时数据传输时,并且同时支持 QoS 协议。

6.包含了数据包和各层的协议架构

在 PCIe 协议规范中规定了采用分层的结构对设备进行设计,并且各个层使用专门的协议架构,同时在各层定义了不同的数据包结构,这样可以使其广泛的应用到其他领域。

7.设备的热插拔和热交换被支持

设备的热插拔和热交换的出现是为了解决客户对计算机系统在很长时间处于开机状态,它主要具有 3 个重要的功能:

- a.不必关闭系统就可以替换发生故障的扩展卡;
- b.修复期间可保证 QoS 和其他服务继续运行;
- c.对故障设备相关的设备可以进行关闭和重启。

8.支持流控制机制

所谓的流控制是为了保证在数据包通过链路发送到接收端之前,发送端必须验证接收端口有充足的缓冲区空间来接受所发生的事[13]。

9.具有低电源消耗和电源管理功能

PCIe 接口总线设备互连的信号较少而且信号的幅度很小,所以它的电源消耗就相对减少很多,它的电源管理是由软件和硬件共同决定的。

10.包含了错误处理功能和先进的错误报告功能

错误可以分成 3 大类,分别制定错误的研究程度:

- 1).可以修正的错误,由硬件处理
- 2).不可以修正的错误—不致命的,由设备中专用的软件进行处理
- 3).不可以修正的错误—致命的,由系统软件处理

通过把错误定义为这些类别,可以将错误处理软件划分成不同的处理程序,去处理所出现的错误;其次可以通过配置空间的 3 个不同区域的配置寄存器来控制 and 报告

错误。

11. 与 PCI 兼容在软件层中

PCIe 接口总线的一个非常重要的特点就是跨平台兼容, PCIe 与 PCI 支持相同的存储器地址空间以及 IO 地址空间, 并且两者具有类似的配置地址空间, 因此两者在软件上保持兼容。

12. 具有引脚少, 可以节约空间, 减少串扰等技术特点

PCIe 接口不用在主板上布置过多的数据线, 相比较于先前 PCI 接口, 将近 75% 的导线数量在 PCIe 接口中被减少, 这样导致传输速度变得更快并且不再需要进行数据同步。由于主板上的布线减少了将近四分之三, 这样能更加方便的实现通过走线数量的增加方式来提高总线宽度, 并且各走线之间的距离就能适当的扩大范围, 变得更宽, 这样就能大幅度的减少各走线相互之间的串扰^[10]。

2.2 PCIe 接口总线的拓扑结构

PCIe 接口相对于 PCI 接口在拓扑结构上有了重大的变化, PCIe 拓扑结构如图 2.1 所示, 其中 PCIe 接口的设备单元具体包含了跟联合体、交换开关和端点三种^[14]。

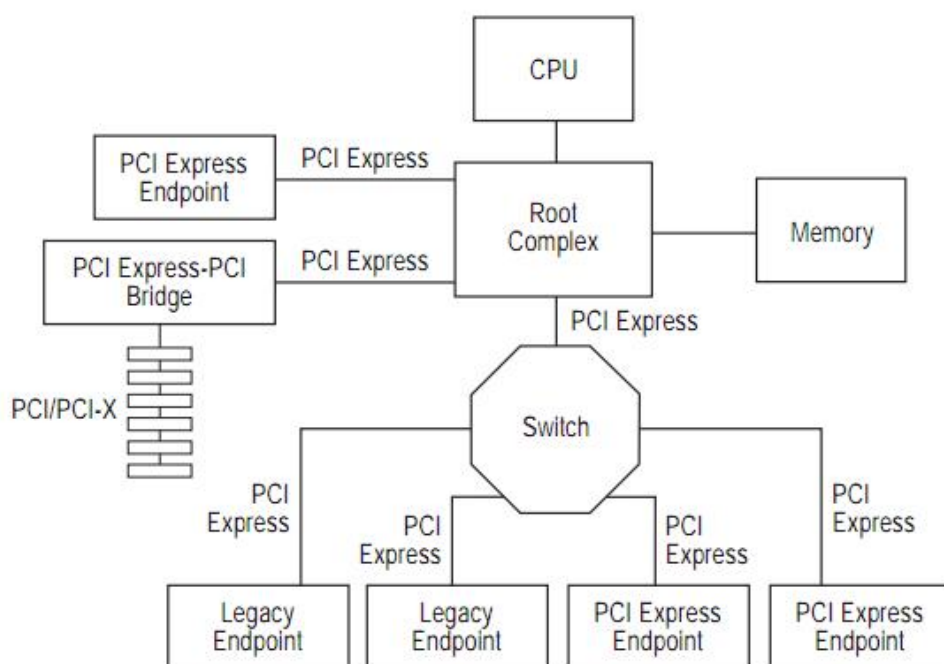


图 2.1 PCIe 接口总线的拓扑结构

如图 2.1 所示, 根联合体 (Root Complex) 的作用是将 CPU 和存储子系统连接到 I/O 设备上, 它可以有单个或多个 PCIe 端口^[14]。在图 2.1 中, 该根联合体包含了 3 个端口, 其中三个端口分别连接着一台端点设备、一个 PCIe-PCI 桥以及一个交换机,

而交换机下层连接 4 台端点设备。当根联合体作为请求者时，它代替 CPU 发起事务请求，其中包括了配置事务请求、存储访问请求、IO 请求以及锁定事务请求。当根联合体作为完成者时，它只对锁定请求不响应。在事务发起时，它的总线号、设备号和功能号都是被初始化为 0 的^[15]。

交换机（Switch）是 PCIe 的交换设备，它相当于由 PCI 架构中的多个 I/O 桥接器组成，对交换机来说，根联合体方向的端口被称为上游端口，其他端口被称为下游端口。交换机逻辑结构图如图 2.2 所示，它的实现方式类似于 PCI 桥的方式，可以通过路由地址的方法转发数据包。交换机可以实现端口仲裁和 VC（virtual channel 虚拟通道）仲裁两种仲裁机制，并且在确定输入端数据包转发至输出端的优先级时可以通过仲裁机制确定，其次交换机支持锁定请求^[16]。

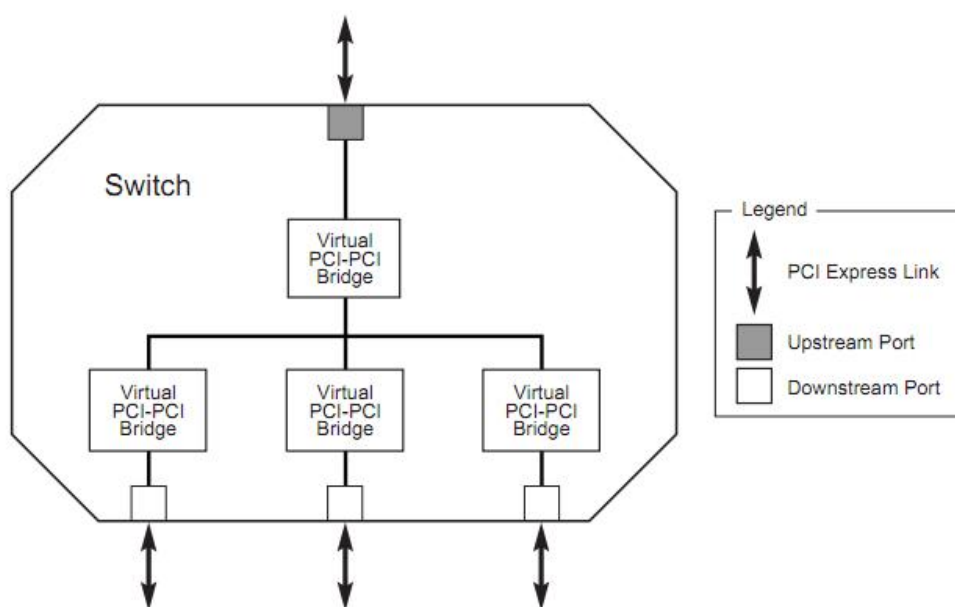


图 2.2 交换机逻辑结构图

端点设备（Endpoint）可以作为 PCIe 事务的请求者和完成者，它与根联合体和交换机等其他设备不同。它是作为外围设备而存在的，类似的有以太网、USB 等。如上图 2.1 所示，PCIe 存在两种端点：PCIe 端点设备和传统的端点设备（Legacy Endpoint）。作为传统的端点时，它不仅支持 IO 事务，而且当其作为完成者时，可以支持锁定事务；其次当传统端点有中断能力时，它可以利用消息请求来支持生成传统的中断生成。PCIe 端点必须在可预取存储地址空间内支持 64 位存储寻址能力，尽量将它们的不可预取存储地址空间映射到 4GB 范围之内。两种类型的端点都实现了 type0（类型 0）配置头，并且作为完成者对配置事务做出响应^[14]。每个端点的初始化可以通过设备 ID 来确定，其中设备 ID 包括了总线号、设备号和功能号。假如在一条总线上的话，则端点总为设备 0。

2.3 PCIe 接口总线的链路

PCIe 接口总线中物理连接两台设备的称为 PCIe 接口总线的链路。一般情况下 PCIe 链路包含了两个低电压差分驱动信号的双单工通路^[17]，其中每条 PCIe 链路的点对点通路数包括了 x1、x2、x4、x8、x12、x16 以及 x32。同等的数目通道在链路的各个方向上被支持。链路两端的设备在硬件初始化过程中，初始化工作会自动完成，其中包括链路的带宽以及工作频率的初始化。PCIe 链路如图 2.3 所示：

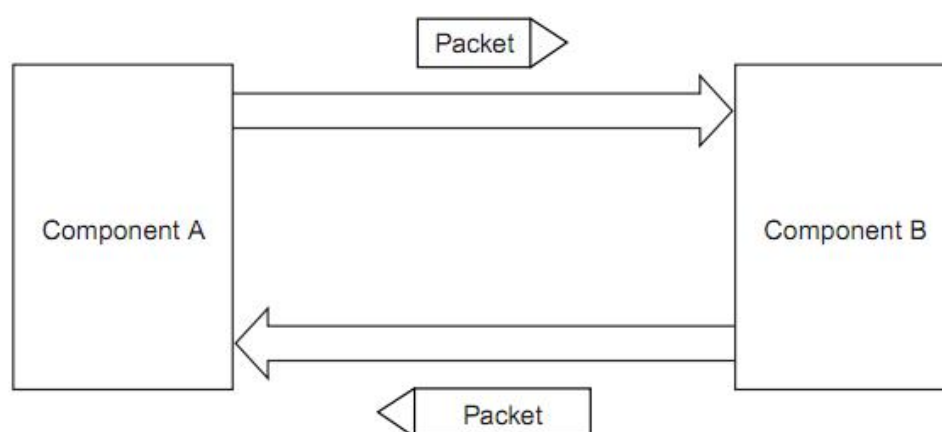


图 2.3 PCIe 链路

2.4 PCIe 接口总线的层次结构

跟 TCP/IP 网络模型类似，如图 2.4 所示：在 PCIe 接口总线协议中定义了底层的物理层（Physical Layer）、中间层的数据链路层（Data Link Layer）和最上层的事务层（Transaction Layer）三层结构。每一层都被分为两部分：一部分用来处理 outbound（发送端）信息，另一部分处理 inbound（接收端）信息。

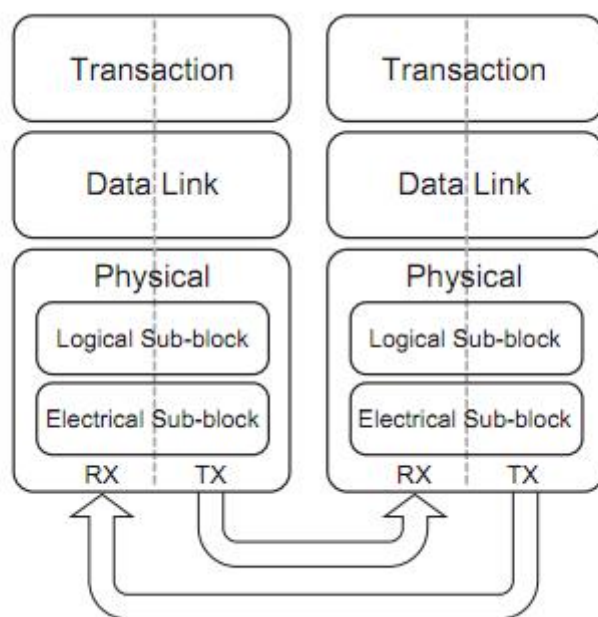


图 2.4 PCIe 接口总线的层次结构图

PCIe 设备互相之间的通信是通过数据包的形式来实现的，在发送端：在事务层形成数据包的内容，其内容是从设备核心和应用程序所获取的；在数据包到达数据链路层时，在该层添加了接受设备检错时所必需附加的信息；之后通过物理层在对其进行相应的编码，并通过物理层的模拟部分在链路上进行差动发送^[18]。该数据包的传送是通过其相邻接收设备的可用链路通道来实现的^[18]。在设备的接收端：数据包的传送与发送端刚好相反，设备通过物理层进行解码得到数据包的内容，并将其发送至上一层；数据链路层将接受到的数据包进行相关的校验查错，要是数据包正确，则将其发送至上一层即事务层，最后通过设备核心和应用程序对转换而来的数据包进行相应的操作。数据包经过各个层所包含的信息如图 2.5 所示：

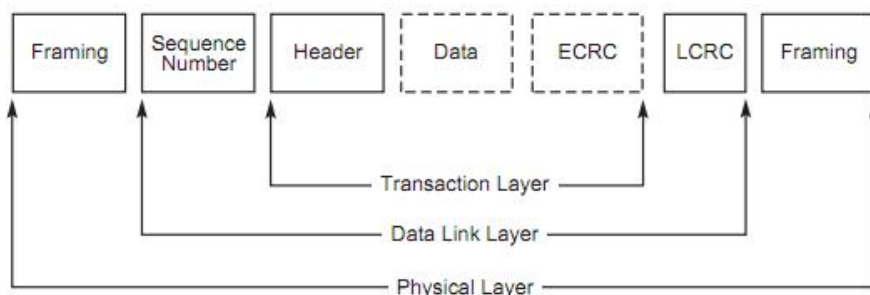


图 2.5 数据包结构

2.4.1 事务层结构

事务层是 PCIe 接口总线的顶层，该层次是将 PCIe 设备核心层的数据请求转换为

PCIe 总线事务^[19]，其主要作用是进行数包的据封包和解包，主要功能包括了配置和初始化功能，即对处理器和器件的链路配置信息进行存储和管理；以及对物理层器件的带宽和频率所确定的链路容量进行存储；还包括了事务包的发生和处理功能，即依旧设备核心层的请求生成读或者写请求的事务包，并且将其传输到链路层；或者是为设备核心层翻译接收到的事务包请求，并将接收到的响应包翻译成设备核心层可以读懂的信息发送至设备核心层。

PCIe 接口相比较于 PCI 接口新增加了消息事务，其他的事务与 PCI 一样包含了存储器事务、IO 事务、配置事务。将请求者和完成者相互之间完成的信息交互所要经过的一系列的过程定义为事务^[10]。PCIe 将事务分为三种，如表 2.1 所示：

表 2.1 事务数据分类

事务类型	分类
存储器写	报告事务 (Posted Packet)
消息	
带数据的消息	
存储器读	非报告事务 (Non-Posted Packet)
I/O 读	
I/O 写	
配置读	
配置写	
带数据的完成	完成包 (Completion)
没有数据的完成	

报告事务与非报告事务之间的区别是，当请求者发送一个事务请求数时，不需要返回完成请求为报告事务；而当请求者发送一个事务请求时，需要接收返回完成请求为非报告事务；完成事务指的是对请求事务的完成，返回完成包。

1. 事务包 (TLP) 的结构

在 PCIe 协议出现之前，一些早期总线协议（例如 PCI 总线协议）容许传送的数据包可以不定义大小，甚至可以无限大^[20]，只有当其数据包传送完毕时才能对有效载荷边界进行确定，而且可能由代理发出信号进行事务结束（例如，在写入时目标断开连接或者在读期间有抢占的发起者），这样会产生事务传送不完整，只是将部分事务进行了传送，当出现这种情况时，如果出现意外，计算和发送覆盖整个有效载荷的校验以及循环冗余校验会变得异常困难。

而与之不同的第三代总线协议 PCIe 总线协议来说，每个 PCIe 数据包都有已知的大小和格式，并且在每个 DLLP 和 TLP 数据包开始位置的数据包头中包含了数据包

的类型和其他类型的定义是否存在。通过数据包的类型定义每个数据包字段的大小以及访问方式，其中也包括了固定大小的数据包。图 2.6 展示了 TLP 的结构图：

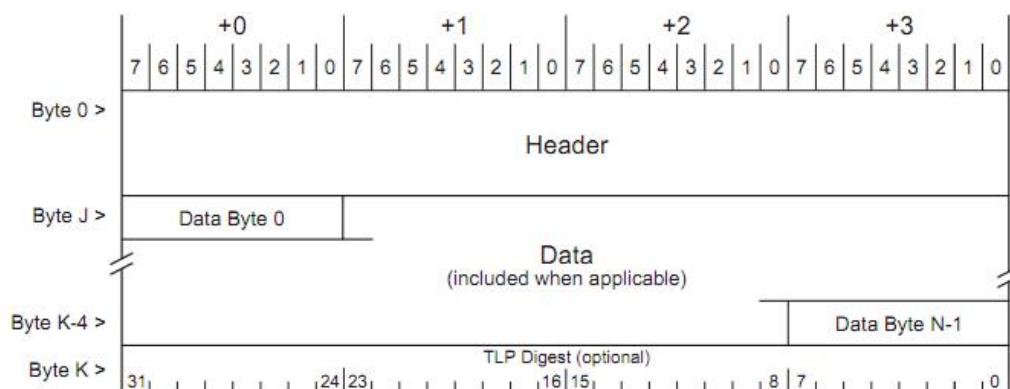


图 2.6 TLP 的结构图

TLP 事务数据包由包头（Header）、数据载荷（Data Payload）和摘要（Digest）三部分组成。其中包头是必须的，也是最为重要的部分，是整个事务包的最重要的部分，它的大小分为两种，分别为 3 个双字（DW）或者 4 个双字（DW），包头的作用是对数据包进行管理和控制。数据载荷的长度是根据包头中的定义的类型而变化的，也可以为空，用于存放数据信息。摘要是可选字段，当其存在时表示 32 位的 ECRC 校验。表 2.2 详细介绍了 TLP 数据包所有组成部分的含义：

表 2.2 TLP 数据包的各部分用途

TLP 数据包的组成部分	用途
包头（Header）	大小为 3DW 或者 4DW（12 或者 16 个字节）格式随类型的变化而不同，可以定义的事务参数包括： 1.事务类型 2.预期的接受者地址，ID 等 3.传送的数据大小（前提是设置了有数据），字节允许 4.顺序属性 5.缓存一致性属性 6.流量类别
数据（Data）	可选字段，是 0-1024 个双字有效载荷，应用字节允许可以进行限制，用来确定字节的地址和字节传送的大小
摘要（Digest）	可选字段。如果存在，大小总是一个双字。用于端到端 CRC（ECRC）和数据中毒（data poisoning）

通用 TLP 头字段是整个事务包中最关键的部分，它不仅定义了 TLP 有效数据载荷的大小，和访问方式。还定义了事务包的流量类型以及其他属性，包头（Header）的结构如图 2.7 所示：

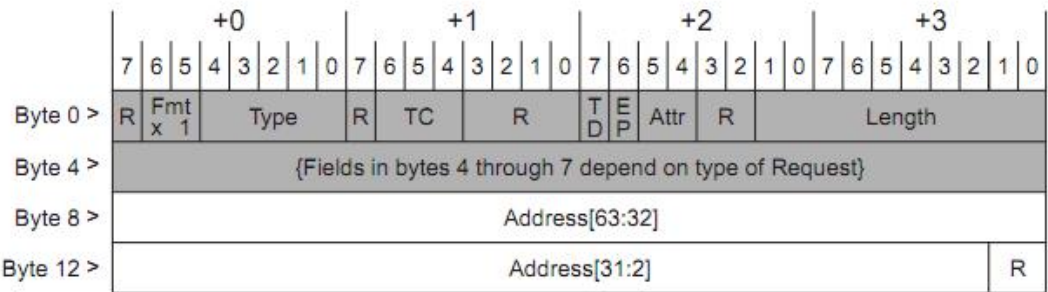


图 2.7 包头（Header）结构

对于整个包头（Header）各个字段的详细用途以及大小如表 2.3 所示，其中标注为“R”的字段为保留字段，默认为 0。

表 2.3 包头（Header）的详细结构

头字段	头位置	用途
R	字节 0 的比特 7 字节 1 的比特 7 字节 1 的比特 3:0 字节 2 的比特 3:2 字节 15 的比特 1:0	保留字段。
格式（Fmt）[1:0]	字节 0 的比特 6:5	这两个比特编码有关头大小以及数据有效载荷是否该 TLP 的一部分等信息： 00b 3DW 头，没有数据 01b 4DW 头，没有数据 10b 3DW 头，有数据 11b 4DW 头，有数据
类型（Type）[4:0]	字节 0 的比特 4:0	这 5 个比特编码是用来定义事务的访问方式和包头的大小等。
流量类别（TC）[4:0]	字节 1 的比特 6:4	这 3 比特用来编码 TLP 和与其相关的完成数据包的流量类别 000b 流量类别 0（默认） 0001 流量类别 1 ...

表 2.3 包头 (Header) 的详细结构 (续)

头字段	头位置	用途
流量类别 (TC) [4:0]	字节 1 的比特 6:4	111b 流量类别 7
摘要字段使能 (TD)	字节 2 的比特 7	要是定义为 1, 一个可选的双字 TLP 摘要字段包括在该 TLP 中, 并且 ECRC 值包含在该 TLP 中。 一般规则: 每个接收器必须要检测摘要字段的存在 1.TD=1, 但没有摘要字段的 TLP 作为畸形 TLP 处理。 2.如果某设备支持 ECRC 校验并且 TD=1, 则必须执行 ECRC 校验。 3.如果在最终目的地, 某设备不支持 ECRC 校验 (可选), 则该设备必须忽略此摘要。
中毒使能 (EP)	字节 2 的比特 6	如果设置为 1, 尽管允许正常事务完成事务, 但伴随此事务的数据应该被认为是无效的。
属性 (Attr)	字节 2 的比特 5:4	1.比特 5 灵活的顺序 当设置为 1 时, 对于此 TLP, 允许使用 PCI-X 灵活顺序, 当设置为 0 时, 则使用 PCI 的严格顺序。 2.比特 4 不侦测 当设置为 1 时, 表明请求者对于此 TLP 不存在主机缓存一致性问题。当设置为 0 时, 要求进行 PCI 类型侦测保护。
长度 (Length) [9:0]	字节 2 的比特 1:0 字节 3 的比特 7:0	TLP 数据有效载荷的传递大小, 以双字为单位。最大传送大小是 10 比特, 210=1024DW (4KB), 对应的编码: 00 0000 0001 = 1DW ... 11 1111 1111b = 1023DW 00 0000 0000b = 1024DW
数据载荷	字节 4-7	随 TLP 类型不同而不同
地址	字节 8 到字节 15	地址也是随 TLP 类型不同而不同

2.事务包（TLP）的类型

对于 PCIe 接口总线协议来说，它包含了存储器地址空间，I/O 地址空间，配置地址空间以及消息地址空间四种不同的地址空间，消息地址空间是 PCIe 接口总线协议所特有的地址空间。如表 2.4 所示，详细介绍了 PCIe 接口总线的地址空间和事务类型。

表 2.4 PCIe 接口总线的地址空间和事务类型^[21]

地址空间	事务类型	用途
存储器	读、写	从系统存储器映射的某一单元中读取数据或者传送数据至系统存储器映射的某一单元。
I/O	读、写	从系统 I/O 映射的某一单元中读取数据或者传送数据系统 I/O 映射的某一单元。PCIe 的 I/O 地址分配给传统的设备。不允许对真正的 PCIe 设备进行 I/O 寻址。
配置	读、写	从 PCIe 设备空间的某一单元中读取数据或者传送数据至 PCIe 设备配置空间的某一单元。与在 PCI 系统中相同，使用 4KB 的 PCIe 配置空间，利用配置来发现设备的功能、编程即插即用特性并检查状态。
消息	基本、厂商专用	提供带内消息和事件报告（不占用存储器或者 I/O 地址资源）。

2.4.2 数据链路层结构

数据链路层（Data Link Layer）是 PCIe 接口总线的中间层，它的主要作用是进行链路管理，保证数据传输的完整性^[22]，即在两台设备之间通过 LCRC 校验确保 TLP 的正确传输。

1.数据链路层数据包（DLLP）结构

相对于 TLP 数据包格式，DLLP 的数据链路层包格式比较简单，它只是用于邻近层次互相之间的通信，并没有携带其他的目标信息。DLLP 结构如图 2.8 所示：

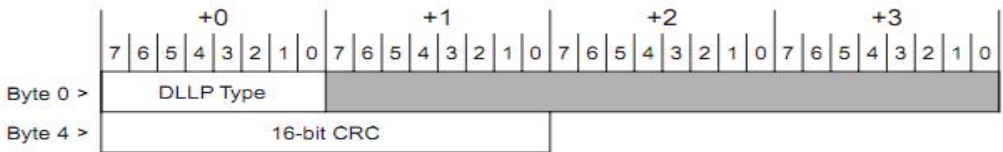


图 2.8 DLLP 结构图

从图 2.8 中可以看出, 数据链路层数据包 (DLLP) 主要包含了以下几个部分:

(1) 字节 0 是 DLLP 类型字段主要定义了 DLLP 的类型, 也就是说定义了 DLLP 的用途。

(2) 字节 1 到字节 3 里面的内容是随着字节 0 (即 DLLP 类型) 的变化而不同。

(3) 字节 4 到字节 5 主要讲的是根据 DW 核心内容计算的 16 比特 CRC, 附加在核心之后。

2. 数据链路层数据包 (DLLP) 类型

在数据链路层数据包 (DLLP) 中, 数据包的类型是数据链路层的关键部分, 通过具体编码的不同定义了数据包的作用及用途, 如表 2.5 所示:

表 2.5 DLLP 数据包类型

DLLP 类型	类型字段编码	用途
ACK (TLP 确认)	0000 0000b	TLP 传送的完整性
NaK (TLP 否认)	0001 0000b	TLP 传送的完整性
PM_Enter_L1	0010 0000b	电源管理
PM_Enter_L23	0010 0001b	电源管理
PM_Active_State_Request_L1	0010 0011b	电源管理
PM_Request_Ack	0010 0100b	电源管理
厂商专用	0011 0000b	厂商
InitFC1-P	0100 0xxx b	TLP 流控制
InitFC1-NP	0101 0xxx b	TLP 流控制
InitFC1-Cpl	0110 0xxx b	TLP 流控制
InitFC2-P	1100 0xxx b	TLP 流控制
InitFC2-NP	1101 0xxx b	TLP 流控制
InitFC2-Cpl	1110 0xxx b	TLP 流控制
UpdateFC-P	1000 0xxx b	TLP 流控制
UpdateFC-NP	1001 0xxx b	TLP 流控制
UpdateFC-Cpl	1010 0xxx b	TLP 流控制
保留	其他	保留

2.4.3 物理层结构

物理层 (Physical Layer) 是 PCIe 层次结构中的最底层, 它主要用于连接数据链路层和链路, 决定了 PCIe 接口总线的点对点串行传输、可配置带宽和热插拔等物理特性。在物理层中, 包含了驱动和锁相环、串转并、并转串等所有与接口操作相关的

所有电路。其次物理层是由逻辑物理层和电气物理层共同组成的。逻辑物理层是用来处理 8b/10b 编码，以及通过数字逻辑处理对数据包进行处理；电气物理层是由各通道的差动驱动器和差动接收器组成，同时也是连接物理层和链路的模拟接口^[23]。

2.5 UVM 验证方法学

2.5.1 UVM 验证平台的基本结构

对于一个完整的验证平台，它不仅需要模拟 DUT 的所有真实情况，给出预期所要达到的结果，而且能够判断 DUT 的输出结果是否可以达到预期效果，给出可靠的覆盖率信息等。典型的 UVM 验证平台的基本构成如图 2.9 所示：

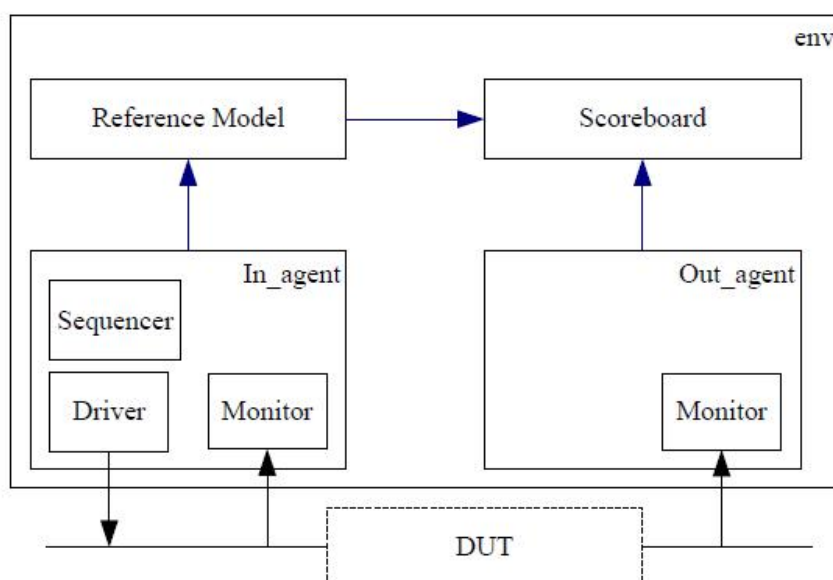


图 2.9 UVM 验证平台结构图

如图 2.9 所示，为 UVM 验证平台的基本框架图，对于其内部通信是通过事务级建模（TLM）来实现的，平台组件是通过事务级进行传输的^[24]，其中每个验证组件都有其验证平台中的特定功能：

DUT（design under test）：待测设计，即要验证的整个设计，DUT 是用 Verilog 或者 VHDL 所编写成的时序电路。

Driver：driver 是整个验证平台数据流的源泉^[25]。在整个 UVM 验证平台里，所有的 driver 都要从 uvm_driver 派生。它的主要功能是向 sequencer 索要 sequence_item(transaction)，并且通过驱动把 sequence_item 里的信息加载到 DUT 的接口上，这等价于完成了信息的转变，即将 transaction 级别信息转变到了 DUT 能够接受的 pin 级别的信息。其中 sequence_item 也是派生自 uvm_sequence_item。

Monitor：监视器是对 DUT 信号进行采样但不驱动它们的被动实体^[26]，在整个

UVM 验证平台中，所有的 monitor 都要从 uvm_monitor 派生出。Monitor 所做的工作恰好与 driver 相反，driver 的作用是向 DUT 的 pin 上发送数据，而 monitor 的作用是从 DUT 的 pin 上接收数据，并且将接收到的 pin 级的信息转换成 transaction 级别的信息，最后将转换后的 transaction 级别的信息发送给 scoreboard，供 scoreboard 进行比较。

Sequencer: 在 UVM 验证平台中，所有的 sequencer 都要从 uvm_sequencer 派生出。sequencer 的主要功能对 sequence 进行组织管理，当验证开始时，它相当于媒介将 sequence 生成的 sequence_item 转发给 driver。

Reference Model: reference model 是从 uvm_component 直接派生的，是用高级语言编写的与 DUT 功能相似的模型，它的功能跟待测设计的功能是一样的并且一定是正确的，不然失去了对比的价值。

Scoreboard: scoreboard 都是从 uvm_scoreboard 派生出来的。它的功能是将 reference model 和 monitor 分别发送来的数据进行比较，并且根据比较结果生成相应的文件，用来判断待测设计是否正确。

Agent: 所有的 agent 都是由 uvm_agent 派生出的。它的主要功能是将 driver、monitor、sequencer 等组件封装在一起，便于重用和管理。

Env: 在 UVM 验证平台中，所有的 env 都由 uvm_env 派生，env 的作用是将验证平台中用到的确定不变的 component 都封装在一起^[27]。这样做的优势是当在跑不同的 case 时，只需要在 case 中实例化一个 env 即可。

2.5.2 UVM 验证方法学的特点

作为新一代验证方法学的标准，UVM 验证方法学继承了 VMM 和 OVM 的优点，成为业界第 1 个通用、开放的验证方法学^[1]，它采用了层次化的验证结构，其内部包含了如 phase、sequence、objection 等机制，这些特点有利于搭建高效的验证平台。本文的主要内容是通过 UVM 及 VIP 技术设计搭建验证平台，对 X 型微处理器的外设 PCIe 接口进行功能验证，因此本节将对该验证平台相关的 UVM 特点进行描述。

(1) UVM 层次化结构

如图 2.10 所示为标准验证方法的层次化结构，层次化的验证平台，将验证环境抽象为命令层，驱动层等不同层次结构^[28]。

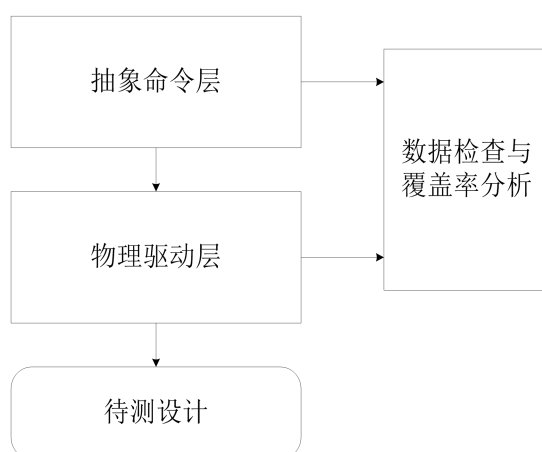


图 2.10 验证方法学分层结构

抽象命令层中一般包含了命令层、功能层、场景层，随着层次的增高，抽象的程度也随着增大，针对不同的 DUT，验证分层可能会有所区别，再具体项目中由验证工程师来决定。验证平台的分层机构由不同的验证组件组成，UVM 通过树形结构，将每个组件以结构的结点模式相互连接，UVM 树形结构如图 2.11 所示。

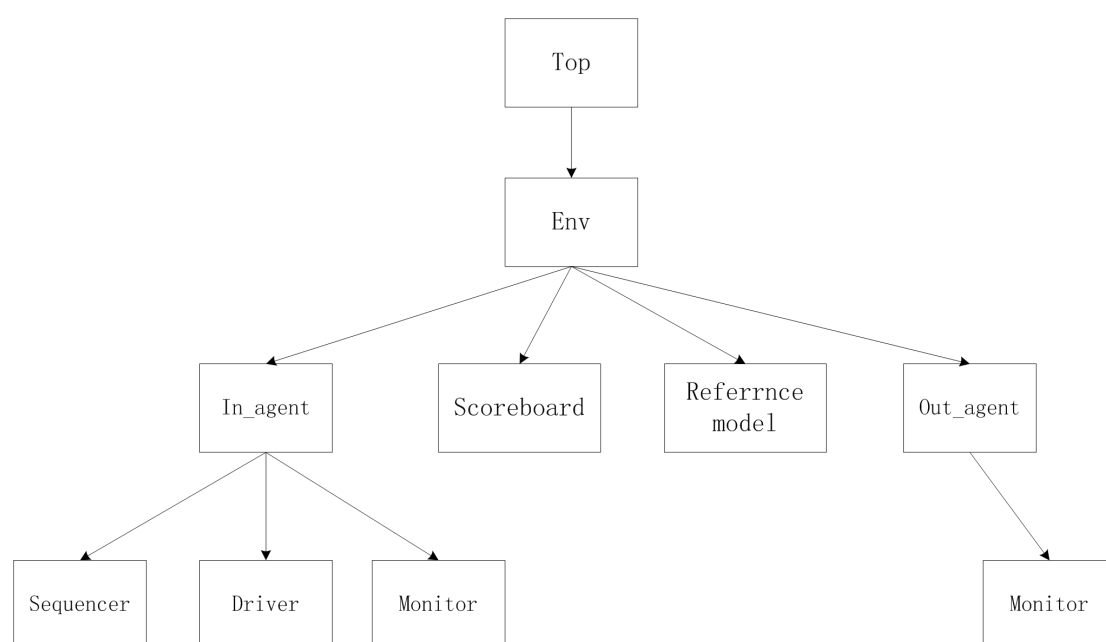


图 2.11 UVM 树形结构

(2) UVM 验证方法学中的部分机制

1) UVM 中的 phase 机制

在 UVM 平台中，所有的组件在应用之前必须进行例化，只有实例化之后，组件之间才可以进行互联，这就导致代码功能的实现受到了代码顺序的影响，而 phase 机制的出现，解决了这一问题，phase 机制将功能相同的代码封装在一起，从而降低了

代码顺序对代码功能实现的影响。图 2.12 列出了 UVM 中的 phase^[29]。

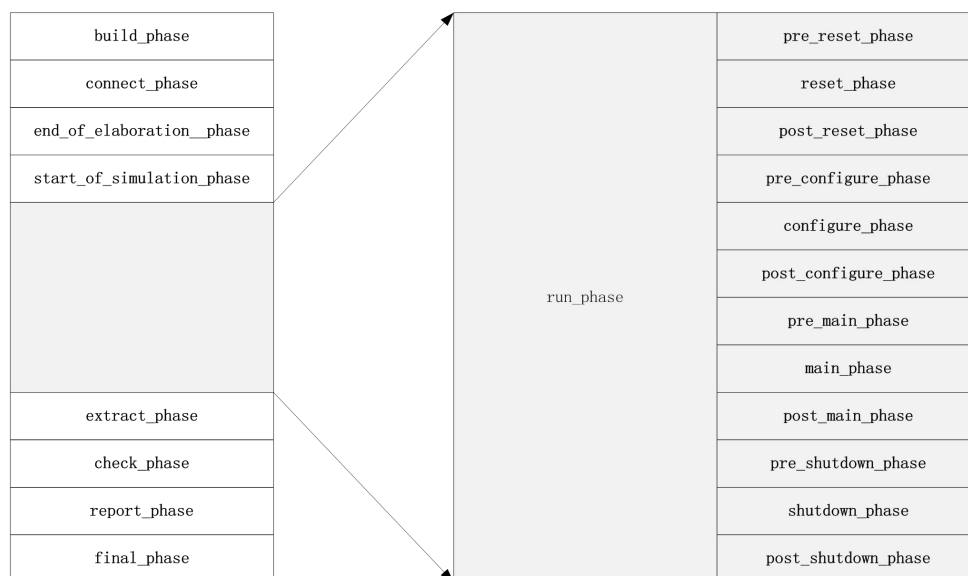


图 2.12 UVM 中的 phase

2) UVM 中的 sequence 机制

sequence 机制是 UVM 验证方法学所独有的，但它并不是作为验证环境中的组件而存在，而是完全独立于整个验证环境的，其本质是一个 uvm_object。UVM 会根据不同的 case 产生不同的 sequence，sequence 作用是用来产生 transaction，通过 sequencer 将 transaction 传送至 driver，driver 再根据其信息产生激励，其次 sequence 另一个作用是控制这个验证平台的开启和关闭。

3) UVM 中的 objection 机制

在 UVM 验证平台中，objection 机制可以通过对验证环境中内部组件的 phase 进程进行控制，实现验证平台的自动开启和关闭^[26]，在实际的验证过程中，当进入某一 phase 的时，此 phase 中的所有的 objection 都将被 UVM 所收集，同时所有的 objection 被实时检测是否已经被 drop 了，当发现 objection 全被 drop 后，此时该 phase 将被关闭，开始进入下一个 phase。\$finish 是用来关闭真个验证平台的，当所有的 phase 执行完后，\$finish 将会被调用。当然在控制验证平台开启关闭时，必须提前提出 raise_objection，在 objection 机制中,raise_objection 和 drop_objection 一定是成对出现的^[30]。

2.6 本章小结

本章主要研究了 PCIe 接口总线协议和 UVM 验证方法学的关键技术，主要包括了 PCIe 接口总线的技术特点、拓扑结构以及各个层的结构以及数据包的结构，UVM

验证方法学的基本架构，以及 UVM 验证方法学的特点，其中包括了验证平台的分层结构以及 UVM 环境运行的部分机制。通过对 PCIe 接口总线和 UVM 环境运行机制的分析研究，给验证工程师在验证功能点提取以及验证平台设计搭建时提供了便利，体现了 UVM 验证方法学的优势。

第三章 基于 UVM 及 VIP 技术的验证平台设计

本章主要针对 PCIe 接口进行基于 UVM 及 VIP 技术的验证平台设计，通过验证策略的制定，搭建出 UVM 验证平台，并且对平台中的组件实现进行详细的说明。

3.1 VIP 组件

VIP (Verification Intellectual Property)，即验证知识产权模块，对于大多数公司的 VIP 来说，都是自己在长期的项目经验中，自己写的 VIP，但是随着时间的推移，为了更加高效的去搭建验证环境，很多公司将 VIP 做成了产品，去推广给其他需要应用的公司，其中很大一部分 VIP 是接口总线的 VIP，本课题就应用到了 Cadence 公司的 VIP 组件去验证 PCIe 接口总线^[31-32]。

对于 Cadence 公司的 VIP，验证工程师可以通过自己制定的规则去选择使能或者非使能，去运行复杂的协议，当一个协议进程出错，VIP 模型发现错误并回收信号，在验证平台中报告错误信息。在复杂的事务交互中，模型回收信号通常是可用的，这个模型帮助计分板比较，使验证更加灵活。

每一种协议的功能都是可配置的，本课题中的 PCIe 协议的功能验证就是通过 VIP 进行配置生成的。其中模型说明概况 (The specification of model architecture SOMA) 配置文件描述了 VIP 模型的规则和其他设备的特性，可以使用 PureView 指令，即运行 VIP 工具指令，为其设备在验证平台结构上去生成 (SOMA) 配置文件和实例化接口模型。

3.1.1 VIP 组件的生成

VIP 通常被使用和连接在平等驱动或者精确循环的逻辑仿真环境中。图 3.1 展示了 VIP 组件生成的各个步骤：

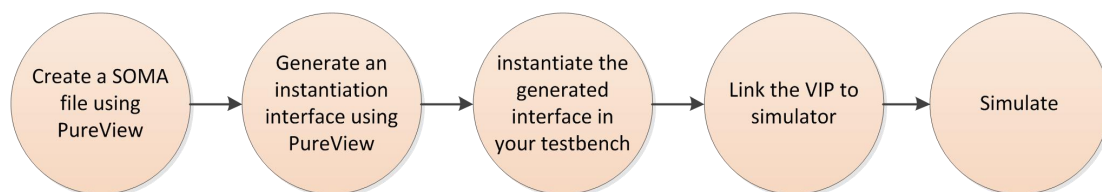


图 3.1 VIP 组件的生成过程图

1. 用 PureView 创建 SOMA 文件

SOMA 文件描述了一个协议模型的特征。这个 SOMA 文件捕捉了一个完整的设备状态，其中包括设备的类型，设备的性能，引脚的接口等，在验证平台的分层系统

中，所有的要素都可以通过 SOMA 进行描述，也可以应用 PureView 图形化工具去创建 SOMA 文件。

2. 应用 PureView 形成实例化接口

采用 SystemVerilog 语言进行实例化接口。

3. 在验证平台上实例化生成的接口

实体在环境中成为了一个实例，通过配置，这个实例可以接收和产生交互，或者行为，相当于 DUT 中的监视器（只在接收事务中）。

4. 通过仿真器链接 VIP（库）

VIP 被提供了以二进制为目标构成的文件，通过链接这个库，VIP 实例通过 API 功能调用，用于与仿真器程序进行交互。这是 UNIX 的同一进程。

5. 仿真

在刚开始的仿真时间为 0 时，配置 VIP 模型实例，定向 SOMA 文件位置和加载 SOMA 文件。当仿真开始进行和信号改变时，VIP 模型更新自己的状态机状态和采用一种基于初速电流状态的适当行动。这个适当的行动包括了发送事务或者标记，报告错误状况和向验证平台提供回调事件。

3.1.2 VIP 的结构框架

VIP 它的结构接近于组合了具有高性能的复杂协议，此外，其中 ACE VIP 模型是基于 SOMA 文件和运行时间配置控制高度可配置的模型。图 3.2 展示了 Cadence VIP 的一般结构：

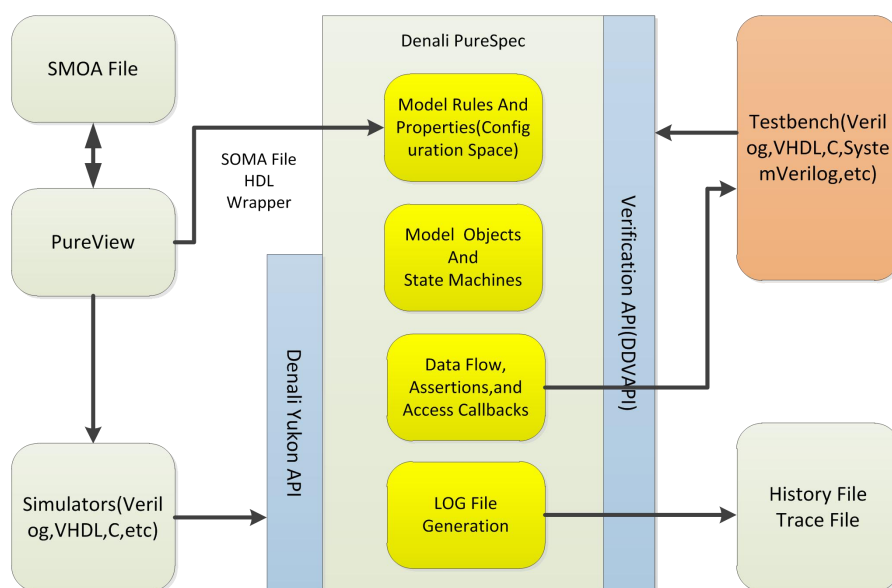


图 3.2 VIP 的一般结构

(1) SOMA （The specification of model architecture）文件

在模型中的每一个设备都有它自己关于它的模型说明概况参数文件去描述它的属性。有关 SOMA 文件的参数在仿真时间为 0 时开始读取。

(2) Configuration Options via SOMA

VIP 模块是高可配的通过 SOMA 参数文件。除此之外，在 VIP 中支持所有的配置选项，通常覆盖了明确的执行选项和可检测性。

(3) 协议规则和检查

对于 VIP 来说,其中最重要的能力是 VIP 有广泛的和完整的协议规则检查。通过 VIP 检查覆盖了所有的被给予的协议说明。在 VIP 中建立和验证的仿真期间上百种运行时间可以被选择去捕获任何潜在的协议违例。

(4) 配置空间

VIP 模型完整的配置空间：

协议配置空间-VIP 定义了所有的协议规范配置寄存器空间，除了协议规范存取方法，这些寄存器在验证平台中是可以使用的。

VIP 配置空间-从协议规范配置寄存器空间分离出来的，VIP 通常被定义成一个配置空间，去许可更多的独立验证平台去控制 VIP 模型。

(5) 模型对象和状态机

在协议规范中 VIP 模型被描述为接近于反应状态机和结构。在每一个模型中，多种多样的队列和状态机被制定，通过验证平台的设置可以操作和使用。

(6) Callbacks(回调)

在 VIP 模型中，有很多重要的数据结构，以及存储器和事务活动。当存取或者事件触发一个回调时在验证平台代码里伴随着相关的信息。可以选择列表中所有回调指向中的需要的回调功能。

(7) 验证仿真接口

VIP 中的验证仿真接口是在验证平台中非常重要的接口，是用于监视 VIP 模型的状态，控制模型的行为，以及产生和定义通信量的关键接口。

3.1.3 VIP 组件的操作模式

在 VIP 有两种主要的运行方式其中包括了：主动方式和被动方式。

在主动方式中，模型的行为类似于一个系统中的组成部件。它可以作为开始或者响应协议的通信，可以很快的去证明这种通信在 DUT 中是否符合要求。

在被动模式中，模型扮演类似于 DUT 的非正式的参考模型用来核对 DUT。但是，它不能产生任何通信。

当运用 VIP 时，必须在每个模型中分别指明它们各自的运行状态，用 PureView 指令去产生 SOMA 和 HDL 文件。除了监视器以外，所有的模型在配置里应该被设置

成主动模式。监视器，它作为 DUT 的一个精确的复制品，同样也是在被动模式中设置成功的模型。

主动方式：如图 3.3 所示（主动方式下的 VIP 模型）。

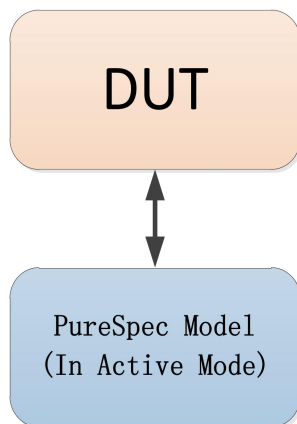


图 3.3 The VIP Model in Active Mode

在主动方式中，随着协议的检查，VIP 模型作为一个总线功能模型(bus functional model BFM)。该运行方式下的模型特征在 SMOA 文件中被描述。

VIP 的验证功能能够被用于产生事务从 BFM 到 DUT。为了初始化和验证主动方式下的模型寄存器和存储空间，通常从验证平台提供的 backdoor access 验证其功能。为了在主动模式里使用 VIP 模型，需要一个 SOMA 文件去描述这个模型和这个主动模式参数设置，连接 DUT 和合适的 VIP 模型连接点。

被动方式：如图 3.4 所示（被动方式下的 VIP 模型）。

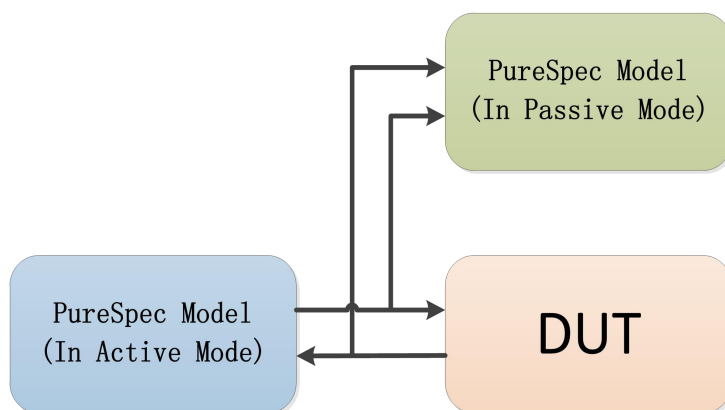


图 3.4 The VIP Model in Passive Mode

被动方式里的模型是一个协议检查者强加于 DUT 上，因为它不能在总线上驱动任何数据，所以在被动模式里作为监视功能。

为了在被动模式里指明模块的功能，需要一个 SOMA 文件配置流入被动模式。连接 DUT 和合适的 VIP 模型连接点。SOMA 文件需要正确的匹配 DUT 以至于它可

以精确的监视 DUT。在被动模式中的 VIP 模型记录 DUT 的输入，并且通过跟踪 DUT 的状况核实其正确性，建立在接收到什么的基础上，和检查 DUT 的传输信号，送出一个指定的信号。

3.1.4 PCIe VIP 数据流

对于本次验证对象 PCIe 来说，Cadence 公司提供的 PCIe VIP 是一个完整的验证 IP，用来验证 PCIe 设计的正确性的。其中主要是验证 PCIe 接口逻辑功能的正确性，它通常被用于事件循环精确的逻辑仿真环境中。

Cadence 公司提供的 PCIe 的 VIP 包括了 PCIe 协议所有的高可配置和灵活的仿真模型层次和设备类型。完整的 PureSpec-PCIe model (PCIe VIP 模型)是围绕着队列去建立的，队列在层与层之间进行数据传输，或者在包的内容中表现为一种变化。PCIe VIP 模型有三种队列：TL User Queue、TL Receive Queue、API Queue。

每一种队列都有各自的一个队列 ID 用来描述队列在哪种操作模式下工作，此外除了上述所提到的队列，还有一部分队列是 PCIe VIP 数据流不能被直接使用的，其中包括了：

- 1.TL Transmit Queue
- 2.TX Completion Queue
- 3.Retry Buffer Queue
- 4.DL Transmit Queue
- 5.DL Receive Queue
- 6.RX Completion Queue

在数据流中验证工程师可以在明确的要点上观察和修改包，它们被提供作为 callback points。验证工程师可以应用 callback points 去阻拦一个包，改变它的特性，在相同的要点重新插入它，这样这个包在它先前的 flow 被重新取回。

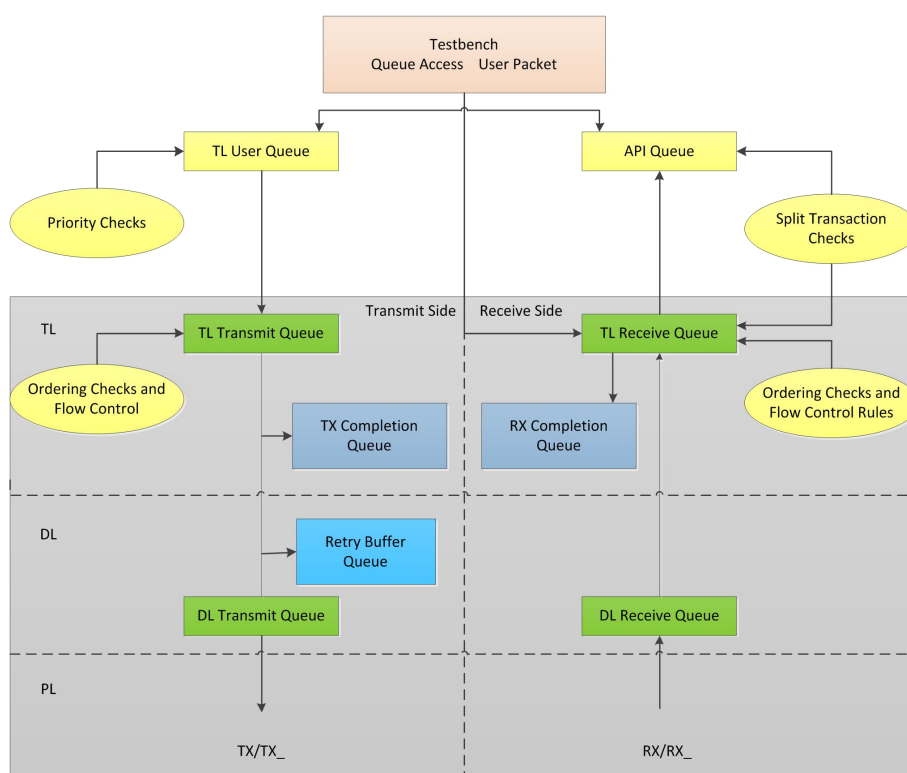


图 3.5 PureSpec Data Flow

1. 发送端（Transmit Side）

一个 TLP（事务包）的形成是通过验证平台（test bench）或者队列入口询问直接加载进 TL User Queue 中，目的是用于传输。所有的数据流通过（the PureSpec model）VIP model 在 Transmit Side 包括下面这些队列中检查。

1) TL User Queue

TL 用户队列是一组 FIFO。每个 FIFO 与不同的延迟值相关联。每个 TLP 可以在其进入 TL 用户队列时指定相应的延迟值，以进入特定的 FIFO。在延迟随着时间过去之后，FIFO 和延迟值为 0 的 FIFO 合并，并且此 0 延迟 FIFO 中的 TLP 已准备好移动到 Transmit 队列。在 TLP 移动到 0 延迟 FIFO 的头部之后，根据适当的协议规则进行检查，经过验证的 TLP 进入 TL 发送队列。

2) TL Transmit Queue

TL 发送队列保存要发送的所有 TL 包，包括用户生成的包和完成包。

验证平台可以验证待测设计（DUT）发送排序规则。这是在发送队列通过启用监视器执行检查 DUT 是否符合适当的排序规则。可以基于 PCIe 排序规则（或基于 ID 的排序规则）和流控制规则来重新排序包传输的顺序。每个 TLP 包在发送队列中花费的最小符号时间是设备特定的参数，并且可以通过 SOMA 文件来配置。默认值为 0，表示没有延迟。

3) Completion Queue

在 TLP 退出发送队列并且如果它需要完成时，完成队列中的条目被插入具有适当的包标签。可以在 TLP 后被发送到 DLL 之前生成 ECRC 并附加到分组的末尾，并且将其添加到重试缓冲区队列，使得可以基于 ACK 和 NAK DLLP 的接收来重新发送或清除它。

4) DL Transmit Queue

在 DLL 处，通过激励号和 LCRC 将 TL 包变换为 DL 包。新形成的 DLP 被加载到 DL 发送队列中。每个分组在 DL 发送队列中花费的最小符号时间是设备特定的参数，并且可以通过 SOMA 来配置。默认值为 0，即没有延迟。

在 PL（物理层）的请求下，DLL 将 DL 发送队列中的 DLP 传递给 PL。在 PL 处，通过在开始处应用 STP 符号并在 DLP 的结束处应用 END 或 EDB 符号来将 DLP 变换为 PLP。然后，在链路上处理和传送 PLP。

2. 接收端（Receive Side）

由验证平台或队列 Access 查询形成的 TLP 可以直接加载到 API 的队列或接收队列的分割事务检查。通过接收端的 PureSpec 模型的整体数据流包括以下队列和检查。

1) DL Receive Queue

从链路接收的分组被加载到 DL 接收队列并被处理。

2) Completion Queue

数据包按照上述过程通过 PCIe 结构，并根据指定的路由（ID，地址，隐式）到达其目的地。在目的地，如果接收的分组是完成包，则完成队列中的相应请求条目被去除。如果在指定的超时期限后未收到预期完成，则会从完成队列中删除数据包，并且完成超时事件会触发对测试台的回调。

3) TL Receive Queue

接收的 TL 数据包进入接收队列，它是一组 FIFO。每个 FIFO 与不同的延迟值相关联。每个 TLP 可以在其进入 TL 接收器队列时通过指定相应的延迟值来进入特定的 FIFO。在延迟随时间流逝之后，FIFO 与具有延迟值 0 的 FIFO 合并，并且此 0 延迟 FIFO 中的 TLP 准备好退出。在 TLP 移动到 0 延迟 FIFO 的头部之后，针对包括排序规则的适当协议规则进行检查，之后，经验证的 TLP 退出接收队列。

3.Ordering Checks

接收队列通过根据适当的协议规则检查 TL 数据包排序，在接收器端提供排序支持。可以基于 PCIe 排序规则来重新排序分组传输的顺序。验证平台可以验证被测设计（DUT）接收器排序规则。这是在接收队列，通过启用监视器执行检查 DUT 是否符合适当的排序规则。

4.Split Transaction Checks

接收队列还提供拆分事务检查。

5.API Queue

在 TL 完成处理接收到的 TLP 之后，它将 TLP 传递到 API 队列用于进一步处理。当 TLP 进入 API 队列时，它被附加到较早到达的 TLP。所有 TLP 使用逻辑字段。

可以基于 PCIe 流控制规则来重新排序分组传输的顺序。这里需要注意的是 API 层处理此 TLP 所需的时间的详细信息不在 PCIe 规范的范围内，它是设计特定的。此功能允许基于数据包动态指定和控制此时间元素。

6.Delayed Processing

在输入 API 队列之前，必须作为规范要求的一部分发生以下效果，包括 FC 信用更新、发送响应接收到的 TLP 中的错误消息以及状态更新到配置空间（例如，已经记录的 TLP 报头寄存器）。

以下描述了每种 TLP 的延迟以及可能发生的副作用。

(1) CfgRd0，如果在输入 API 队列时指定了延迟，则在指定时间到期后，将从配置空间读取 CplID 中的数据。

(2) CfgWr0，如果在进入 API 队列时指定了延迟，则配置空间寄存器中的值将在指定时间过期后更新。

(3) MRd, IORd，如果在输入 API 队列时指定了延迟，则在指定时间到期后，将从 BAR 存储器读取 CplID 中的数据。

(4) MWrr, IOWrr，如果在输入 API 队列时指定了延迟，则在指定时间过期后将更新 BAR 内存值。

(5) 其他 TLP，指定的延迟将被忽略，TLP 将立即退出 API 队列。

对于 PCIe VIP 模型生成跟上一节所属基本一致，只是在生成过程中选择 PCIe 相关配置即可。

3.2 验证策略的制定

3.2.1 验证设计对象

本课题中所要验证的 PCIe 接口是作为 X 型微处理器的外设，所以在制定验证计划的时候，一定需要对设计对象有明确的认识。整个 MCU 的框架图如图 3.6 所示：

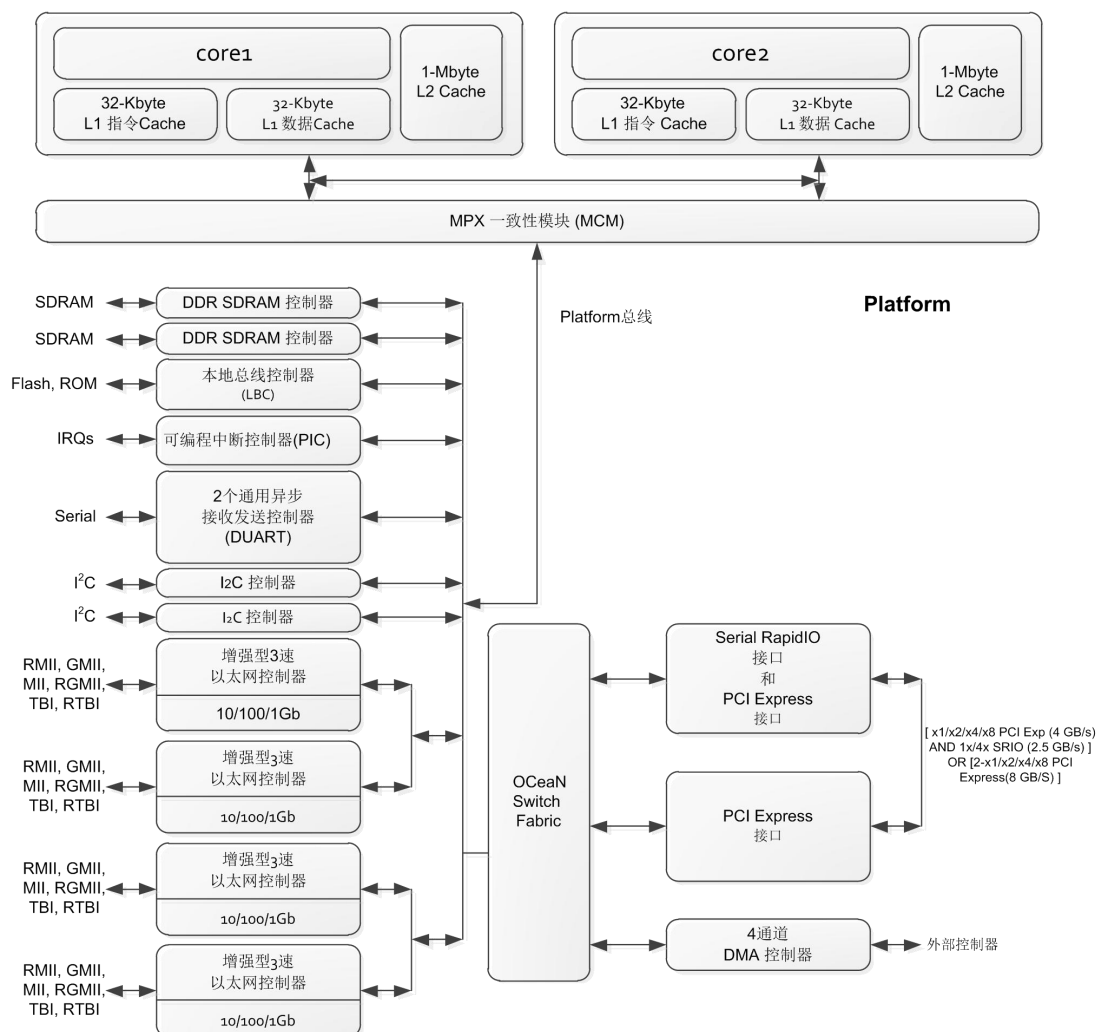


图 3.6 X 微处理器的框架图

如图 3.6 所示,为该课题中微处理器框架图,从图中可以看出该微处理器提供两个实例化的 PCIe 控制器,其次该处理器还包含了 SRIO、以太网等外设结构,本课题只对 PCIe 接口进行验证,因此其他设备接口就不一一进行说明。

详细的有关 PCIe 的结构如图 3.7 所示,X 微处理器包含两套独立的 PCIe 控制器,每个控制器都可以做为 RC 或 EP 使用。串行端口通信的波特率为 2.5Gbps,能支持 X1/X2/X4/X8 四种通道设置。控制器包含了两套独立的 Serdes 模块,其中 PCIe1 独立使用 Serdes1,PCIe2 与 SRIO 共用 Serdes2 (SRIO 只使用 4-7 通道)。PCIe 端口的通道数据通过片上网络结构 (OCeaN) 与 X 微处理器中的片上总线 (MPX) 相连接,实现与其它设备之间的通信。

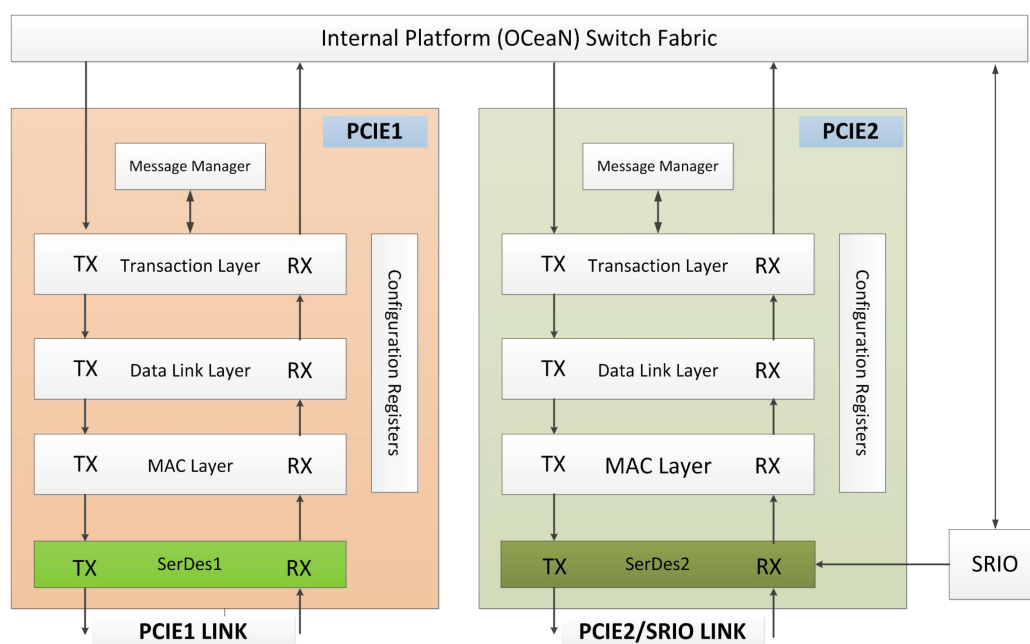


图 3.7 X 微处理器 PCIe 模块框图

在 X 微处理器中，PCIe 接口可作为主控设备与目标设备，可以处理高带宽数据传输。在复位后，PCIe 接口与对接设备进行链路初始化，交换链路宽度与流控信用等信息，当链路初始化成功完成后，控制器才能进行正常的的数据操作。

如图 3.7 所示，在 X 微处理器的 PCIe 内部，设计了 inbound 与 outbound 事务窗口，用来接收与发送事务，同时设计了队列来缓存事务，还有专门的控制逻辑来处理这个缓存管理、总线协议及 tag 位产生。此外，需要使用 memory 块来存取 inbound 及 outbound 数据。

本课题中的 PCIe 控制器可以配置为 root complex (RC) 或 endpoint (EP)，作为 RC 设备时，连接主控 CPU/Memory 子系统到 IO 设备。当作为 EP 设备时表示为作为外设或 IO 设备^[33]。在 RC 模式下，验证人员使用 type1 配置头空间，在 EP 模式下，验证人员使用 type0 配置头空间。

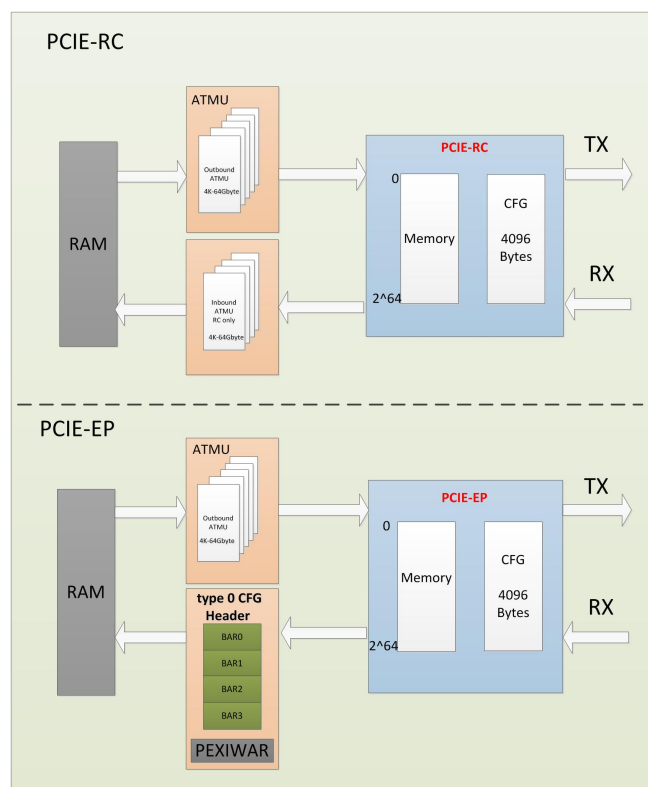


图 3.8 X 微处理器 PCIe 内部实现结构图

作为一个初始化器，PCIe 控制器支持存储器读和写操作，在最大传输带宽 256bytes。另外，当 PCIe 控制器被配置成 RC 模式时，支持配置和 I/O 数据转换。作为一个目标接口，PCIe 控制器接受读和写操作到本地存储空间。当 PCIe 控制器被配置成为 EP 器件配置时，PCIe 控制器接受配置数据转换到内部 PCIe 配置寄存器，信息生成和接收都通过 RC 和 EP 模式支持。锁定数据转换和内部 I/O 转换不被支持。

1. 向外数据转换

内部的平台数据转换到 PCIe 被第一个映射到转换窗口，并且考虑 PCIe 转换被发送。一个来自内部平台的数据转换可以变成一个 PCIe 存储器，I/O，信息或则是数据交换配置。

数据交换可能会出现问题，在小带宽传输，传输类型或者是 PCIe 器件控制寄存器 [MAX_PAYLOAD_SIZE] 要求下。写数据要求或者是 PCIe 器件控制寄存器 [MAX_PAYLOAD_SIZE] 的读要求。控制器执行 PCIe 顺序规则，检查哪一个数据交换会被送往 PCIe 连接。

总的来说，数据交换需要顺序执行，为了使得它们可以从内部平台接收到信息。只有当一个失速状态使得控制器提供 PCIe 顺序规则，才能使数据向外传输。在之后的写数据传输，只要所有的数据可以从内部平台传输，数据就会被 PCIe 传输。为了不发生后写操作，控制器等待完成包返回。为了不发生后读操作，控制器会等待所有的完成包返回，在终止数据交换之前所有的数据回到内部平台。

在复位之后或者是连接条件恢复时,外部数据传输不会发生,除非连接已经成功。软件可以选中 LTSSM 状态寄存器(PEX_LTSSM_STAT),检查连接状态,在外部要求发出时。

2.向内的数据转换

向内的 PCIe 数据转换到内部平台可以第一时间被映射到转换窗口,然后再考虑内部平台转换是否被发送。

当数据向内部平台发送时,数据处理可能会受到小带宽的影响,这也取决于要求的带宽,字节使能,开始、结束地址。控制器执行 PCIe 顺序规则检查可以考虑数据转换到下一个内部平台的条件。

总的来说,数据从 PCIe 连接接收。只有失速状态可以提供 PCIe 顺序到显著的数据交换。为了后续写数据,只要所有的数据可以从 PCIe 连接接收,内部平台的数据和数据转换就算完成。而为了不发生后读转换,控制器就不会向内部发送数据。控制器分裂处理在每 256 字节对齐边界跨越。只要发送的数据返回 PCIe 连接。

该处理器支持的 PCIe 事务如 3.1 表所示:

表 3.1 该处理器支持的 PCIe 事务

PCIe Transaction	Initiator	Target
Mrd	Yes	Yes
MRdLk	No	No
MWr	Yes	Yes
IORd	Yes (RC only)	No
IOWr	Yes (RC only)	No
CfgRd0	Yes (RC only)	Yes
CfgWr0	Yes (RC only)	Yes
CfgRd1	Yes (RC only)	No
CfgWr1	Yes (RC only)	No
Msg	Yes	Yes
MsgD	Yes (RC only)	Yes (EP only)
Cpl	Yes	Yes
CplD	Yes	Yes
CplLk	No	Yes
CplDLk	No	No

3.特征

本课题中的 PCIe 控制器所支持的特征包括：支持 RC 和端点配置、支持 32 位与 64 位地址、在每个 SerDes 中支持 x8, x4, x2, x1 连接、支持访问所有的 PCIe 存储器和 I/O 地址空间、支持后续的处理到 PCIe 和 PCIe 到存储写操作、支持严谨或松散的数据处理顺序规则、PCIe 配置寄存器(EP 模式中 0 类型，RC 模式下的 1 类型)、基线和高阶错误报告、一个虚拟频道(VC0)、256 字节最大有效载荷大小(MAX_PAYLOAD_SIZE)、支持 3 个向内通用目的数据转换窗口和一个配置窗口、支持 4 个向外转换窗口和一个默认窗口、基底流控制管理、支持 PCIe 信息和中断以及接收来自内部平台最多 256 字节数据处理等。

4.接口信号

(1) 输入信号 SD1_RX[0:7]/NSD1_RX[0:7]以及 SD2_RX[0:7]/NSD2_RX[0:7]

它们是串行数据输入，是差分信号对。就 1X 操作模式而言，在 1X/2X/4X/8X 兼容 PCIe 接口下，只有第一条通道可以使用。

(2) 输出信号 SD1_TX[0:7]/NSD1_TX[0:7]以及 SD2_TX[0:7]/NSD2_TX[0:7]

它们是串行数据输出，是差分信号对。就 1x 模式而言，在 1X/2X/4X/8X 兼容 PCIe 接口下，只有第一条通道可以使用。

表 3.2 PCIe 接口信号

信号名	位宽	I/O	信号描述
SD1_RX[0:7]	8	input	数据输入
NSD1_RX[0:7]	8	input	数据输入
SD1_TX[0:7]	8	output	数据输出
NSD1_TX[0:7]	8	output	数据输出
SD2_RX[0:7]	8	input	数据输入
NSD2_RX[0:7]	8	input	数据输入
SD2_TX[0:7]	8	output	数据输出
NSD2_TX[0:7]	8	output	数据输出

3.2.2 验证计划的制定

为了更快更有效率的去验证项目中的待测设计，即 PCIe 接口，采用当前最流行的验证方法之一，基于 UVM 以及 VIP 技术去搭建验证平台，由于公司已有完备的

Cadence 公司的 VIP 产品同时有一定的 UVM 验证平台搭建经验，因此在验证平台的搭建有了明确的目标，搭建一个可复用的基于 UVM 及 VIP 技术的验证平台去验证待测设计。

1.熟悉整个 MCU 的架构，以及 PCIe 协议规范，列出验证功能点。

前期工作主要是熟悉整个 MCU 架构，由于所要验证的 PCIe 接口是 X 型微处理器的外设，不同于单独的 PCIe 接口，在配置该 PCIe 接口时，需要了解它在 MCU 中的地址分配空间以及各个寄存器的配置，因此得花一部分时间去研究分析待测设计。并且列出所要验证待测设计的验证功能点。此外由于该项目中 CPU 是基于 PowerPC 架构即精简指令集架构，因此在进行汇编向量的编写时，要首先对 PowerPC 架构下的指令集熟练应用。

2.掌握 Cadence 公司 VIP 的组件的应用。

本项目中验证的关键点以及优点就是在于 VIP 组件在整个 UVM 平台的使用，因此在验证平台的搭建之前要掌握 PCIe 的 VIP 的组件的生成和应用，正确高效的将 VIP 组件应用到验证平台中。

3.搭建验证平台。

结合待测设计、VIP 组件搭建 UVM 验证平台，明确各个模块的作用以及应用，搭建一个可复用的高效的验证平台。

4.实现验证功能。

通过编写激励，验证待测设计，完成验证功能点的测试，并且与设计人员沟通，修复所验证出的 bug，完成整个设计任务以及验证任务。

3.2.3 验证功能点的提取

对于一个设计来说，验证它的正确性是确保设计正确设计的关键点，而对于验证环节来说，完整的功能测试点则是完成整个验证的关键，一个完整的测试功能点可以保证这个设计的功能都被完整的覆盖，避免出现功能的缺陷。对于该项目中的 PCIe 接口，它即兼容 PCIe 协议规范，又有自己的功能，比如在 MCU 中通过 DMA 最大 256Bytes 从 PCIe 中传输数据等。因此功能点较多，如果不仔细逐项编写并严格审阅各项功能点，很容易出现功能覆盖漏洞^[34]。因此在编写功能点时，先按类分，在按类细分成具体的功能点。PCIe 控制器的测试功能点分为以下几个方面：

(1) PCIe 的事务层功能，其中主要是 transaction 级功能：包括了存储器读写事务，IO 读写事务，配置读写事务，消息请求事务，以及完成事务，带数据完成事务等。注意：其中存储器读写事务包括了 32 位和 64 位读写事务，以及通过 DMA 进行 32 位和 64 位的事务读写的操作。

(2) PCIe 数据链路层功能，其中主要基于信用的流控制功能以及 LCRC 的发送

与接收功能等。

(3) PCIe 物理层功能, 将验证接口分为 X1,X2,X4,X8 四种接口通道情况, 验证各种通道状态下事务是否可以正确完成以及链路状态机跳转功能验证等。

(4) PCIe 控制器寄存器, 涉及到控制器定义的属性和功能, 如其中的电源管理, 中断控制等。

(5) 错误处理功能, 其中包括了可纠正错误 (Correctable Errors) 和非可纠正错误 (Uncorrectable Errors)。对于可纠正错误主要包括 Bad DLLP 和 Bad TLP 可纠正错误, 非可纠正错误中的 Fatal Errors 和 Non-Fatal Errors。

表 3.3 列出了上述事务层的部分功能点:

表 3.3 事务层部分功能点列表

序号	功能点描述
PCIe_1	事务层:Memory MAP 空间寄存器的读写及属性的正确性
PCIe_2	事务层:通过 outbound ATMU 访问配置空间
PCIe_3	事务层:通过 PEX_CONFIG_ADDR/PEX_CONFIG_DATA 访问配置空间
PCIe_4	事务层: type1 配置空间寄存器读写及属性验证的正确性
PCIe_5	事务层:普通 32 位存储器读写事务, 并覆盖上下边界
PCIe_6	事务层:普通 64 位存储器读写事务, 并覆盖上下边界
PCIe_7	事务层:DMA 32 位存储器读写事务, 并覆盖上下边界
PCIe_8	事务层:DMA 64 位存储器读写事务, 并覆盖上下边界
PCIe_9	事务层: 64 位存储器读写事务--预取功能
PCIe_10	事务层:最大数据载荷 256byte 访问 DMA 方式
PCIe_11	事务层:监视处理器访问
PCIe_12	事务层:LAW 窗口与 outbound 窗口映射导致的边界测试
PCIe_13	事务层:outbound 窗口映射冲突测试
PCIe_14	事务层:inbound 边界测试
PCIe_15	事务层: IO 读写事务(IO 空间仅用于兼容合法的设备, 只限 32 位地址空间的 IO 设备 4byte 对齐访问)
PCIe_16	事务层-消息事务: ATMU Outbound Messages PM_Active_State_Nak

表 3.3 事务层部分功能点列表（续）

PCIe_17	事务层-消息事务: ATMU Outbound Messages PME_Turn_Off
PCIe_18	事务层:Set_Slot_Power_Limit
PCIe_19	事务层-消息事务: ATMU Outbound Messages Attention_Indicator_On
PCIe_20	事务层-消息事务: ATMU Outbound Messages Attention_Indicator_Blink
PCIe_21	事务层-消息事务: ATMU Outbound Messages Attention_Indicator_Off

3.2.4 验证环境的构建

在前面的验证计划中已经明确了验证环境通过采用标准的 UVM 架构，同时使用 VIP 组件去搭建完整的验证环境。对于本项目中的验证平台设计主要有三个难点以及关键点：

1.待测设计（DUT）为 SoC 上的一个外设，因此在对 DUT 进行验证的时候，需要通过 CPU 发送指令去配置 PCIe 控制器，其中包括窗口的映射地址，事务包的访问类型、大小等，因此需要对 CPU 的指令的运用非常熟练。

2.本次课题采用的是 UVM 方法学进行验证，其目的是可以使验证过程更加有效率，验证结果更加可靠，通过验证平台的搭建实现待测设计的功能验证，并且进行覆盖率信息的收集。

3.在本课题中的应用了 Cadence 公司已经作为产品应用的验证 IP（VIP），应用 VIP 可以减少大量的编码任务，但是前提是可以正确应用 Cadence 公司的 VIP 到验证平台中。

3.3 PCIe 验证环境架构

由上面章节中的设计对象可知，本课题的 PCIe 接口有两个 PCIe 控制器，并且两个控制器都可以作为 RC（跟联合体）和 EP（终点设备），因此我们的待测设计及 DUT 分为验证 RC（跟联合体）和 EP 两种情况，并且对 PCIe1 和 PCIe2 分别进行作为 RC 和 EP 时进行验证，验证基本框架图如下图所示：

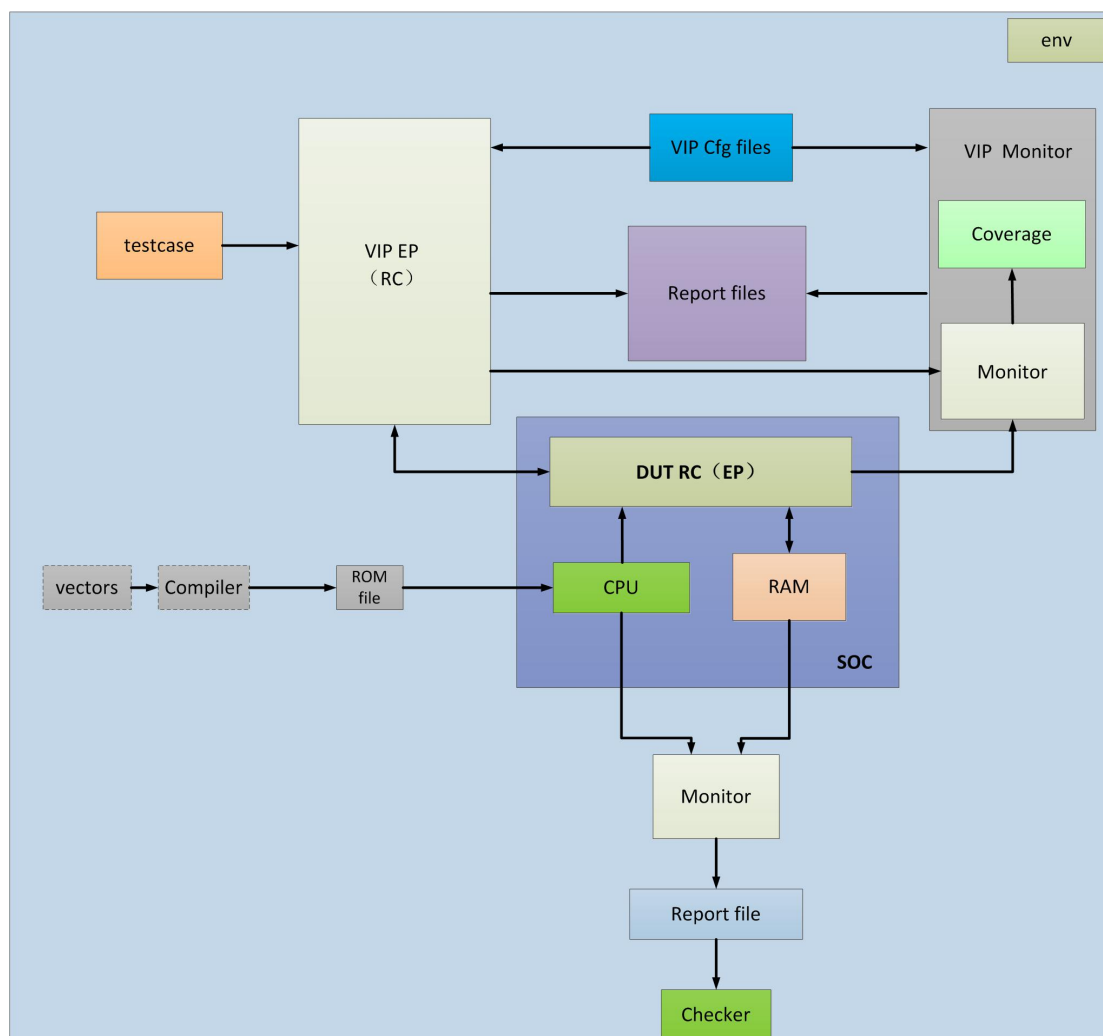


图 3.9 验证平台基本框架图

如图 3.9 所示，为验证平台基本框架图，其中 DUT 是 X 型微处理器上的外设接口，及 PCIe 控制器，由上节验证设计对象描述可知，图中的 DUT 及 PCIe 控制器可以通过 CPU 被配置成 RC，也可以配置成 EP，两种控制器的验证方法相同，组件生成相同，因此此处只对 PCIe 控制器配置成 RC 模式下的情况做简要的描述。

如图所示，其中 testcase 所连接的灰色模块是 VIP 生成 EP (RC) 模块，其中包含了 Agent 模块，将在下一节详细阐述。由前面章节可知本课题中在进行 DUT 的相关配置是必须通过 CPU 去发送相关的指令去配置相关的寄存器，就有了如图所示 SoC 模块。其次如图所示 VIP monitor，该模块主要是作为监视器使用，并进行覆盖率的收集。

1. testcase 模块，这个模块主要是包含所要验证的激励，也就是需要发送的包，其中这个包里包含了事务的类型，地址信息等。

2. VIP EP (RC) 模块，这个模块相当于 PCIe 控制器的高级语言模式，通过配置既可以充当 EP，也可以充当 RC，通过具体的验证要求进行配置调用，该模块还包

含了 Agent 模块，Agent 模块里面包括了一些如 Sequence、monitor、driver 等的基本模块，这些组件是通过 VIP 生成的，在生成组件的时候要通过很多配置，才可以将生成的 VIP 组件应用到验证平台中，图中所示 VIP config files 就是用来配置这些组件，让其正确的应用到验证平台中。

3. VIP monitor 模块也是通过 VIP 去生成的，它不仅包含了 monitor 模块而且还包含了 coverage 模块，monitor 监测 DUT 的输出数据，和 VIP model 发出的数据，并将其发送至 coverage 模块，进行覆盖率的收集。

4. SoC 模块，图中该模块包含了 CPU、PCIe 接口（DUT）以及 RAM，前面章节介绍过，由于本课题的 PCIe 接口是某 CPU 的外设，因此在对 DUT 进行相关操作时，需要通过 CPU 发送指令去配置 PCIe 接口控制器的相关寄存器，比如验证对象是 PCIe1 还是 PCIe2，是需要去配置一些相关寄存器的（本课题的 PCIe 有两个控制器，因此在验证时需要进行配置相关寄存器的位，在后面章节进行详细的介绍）。RAM 的作用是配置相关寄存器所存放的数值，CPU 的 monitor 和 checker 是监视和比较是否和 RAM 中的数值一致。

5. SOC 外的模块 vector，是配置 DUT 相关寄存器的激励文件，通过编译生成 CPU 可识别的机器码，CPU 在去做动作配置 DUT，它和 testcase 里的激励是配合使用的。

6. 框图中的 Report file 是一些结果日志文件，对整个验证进行记录作用，其中还包含一些通过 Perl 脚本生成出来的一些有用信息的记录，方便检查 bug。

3.4 验证平台子模块的实现

本次课题应用的 UVM 验证方法学进行验证平台的搭建，因此采用了 UVM 方法学的分层设计方法进行验证平台的设计，并且严格按照已制定的验证规范进行平台的搭建。上一小结已经给出了验证平台的基本框架，并且做了简要的说明，下面按验证层次对每个模块进行详细的介绍。UVM 验证平台框架图如图 3.10 所示。

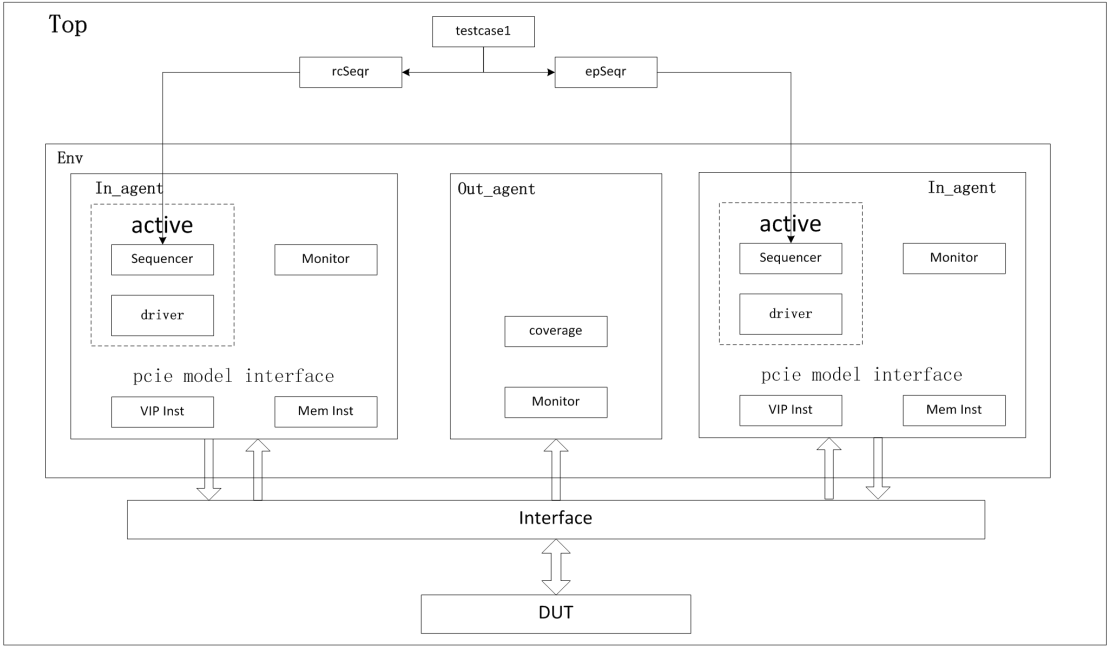


图 3.10 UVM 验证平台框架图

验证系统中的各个验证组件与 UVM 基本类的对应关系如表 3.4 所示，在实现整个验证系统时，所有相关的组件都要统一从 UVM 基类中派生，从而可以应用基类所定义的属性和方法。

表 3.4 验证组件与 UVM 基本类的对应关系

验证组件	对应 UVM 中的基类以及其功能
测试用例	派生于 <code>uvm_test</code> ，其作用是用来产生仿真场景。
验证组件环境	派生于 <code>uvm_env</code> ，其作用是管理验证环境中的各个组件，并且把验证平台用到的派生于 <code>uvm_env</code> 固定不变的所有 <code>component</code> 组件都集成封装在 <code>env</code> 中，并且完成对所有组件的配置工作。
代理器类	派生于 <code>uvm_agent</code> ，其作用是用来封装驱动器、监视器以及激励发生器。
事务类	派生于 <code>uvm_sequence_item</code> ，其作用是用来产生随机可约束的事务类。
驱动器	派生于 <code>uvm_driver</code> ，其作用是用来驱动 DUT。
激励发生器	派生于 <code>uvm_sequence</code> ，其作用是用来产生各种事务
监视器	派生于 <code>uvm_monitor</code> ，其作用是监视 DUT 输出信号，并且转换事务形式。
覆盖率收集器	派生于 <code>uvm_component</code> ，其作用是用来收集覆盖率信息。

3.4.1 顶层 top

验证环境的顶层 top 层，其目的是主要用于产生时钟和复位信号、例化 DUT、声明接口 interface 以及不同验证情景 test 对应的 UVM 验证平台的启动等；其中，DUT 是待验证的系统，也是我们验证的关键点之一，由于该项目的 DUT 是 X 型微处理器的外设，因此在时钟复位时要遵循 X 处理器的复位时间以及周期，其次为了保证所搭建的验证平台在功能仿真和后仿真过程中的可重用性，这里选择采用灰盒的验证方式，即允许验证过程中调用 DUT 内部部分被选中的信号。

3.4.2 环境类 env

env 是派生于 uvm_env，env 相当于重用组件的大容器，大部分验证组件重用都出现在 env 层次上^[26]；它的作用是把验证平台用到的固定不变的所有 component 组件都集成封装在 env 中，并且完成对所有组件的配置工作。是一个灵活性高、适用性较好并且可扩展的组件。当验证人员需要运行不同的 case 时，只需要在 case 中实例化一个 env 即可，这样大大提高了验证平台的可重用性。

在本次课题验证平台中，env 实例化了 VIP model，其中包括 RC 和 EP VIP model，作为参考模型与 DUT 进行比较，其次还实例化了 driver、monitor、scoreboard 等。部分 env 模块程序实现如图 3.11 所示。

```
function new(string name = "cdnPcieUvmUserARcPEpEnv", uvm_component parent = null);
    super.new(name, parent);
endfunction : new

virtual function void build_phase(uvm_phase phase);
    string inst_name;
    `uvm_info(get_type_name(), "In Build phase", UVM_LOW);
    super.build_phase(phase);

    // Active RC
    uvm_config_db#(int)::set(this, "activeRc", "is_active", UVM_ACTIVE);
    uvm_config_db#(int)::set(this, "activeRc", "portType", PCIE_PORT_RC);
    uvm_config_db#(int)::set(this, "activeRc.sequencer", "count", 1);
```

图 3.11 env 部分程序图

3.4.3 接口 interface

在验证平台中将 monitor 和 driver 跟 DUT 链接需要经过 interface 接口，通过接口实现验证环境与待测设计的通信。接口的使用把模块的传输从其他功能模块中区分出来，是模块直接地连接变得更加清晰，这样设计可以使得减少模块之间连接产生的错

误，并且可以提高验证平台设计的可重用性。如图 3.12 所示为 DUT 与 VIP model 接口连线图。其中 monitor 即监视器用来记录的 RX 和 TX 端口 DUT，并通过跟踪装置的状态（链路状态、配置、流量控制等）基于已接收到的内容，检查设备是否为给定状态发送相应的信息。

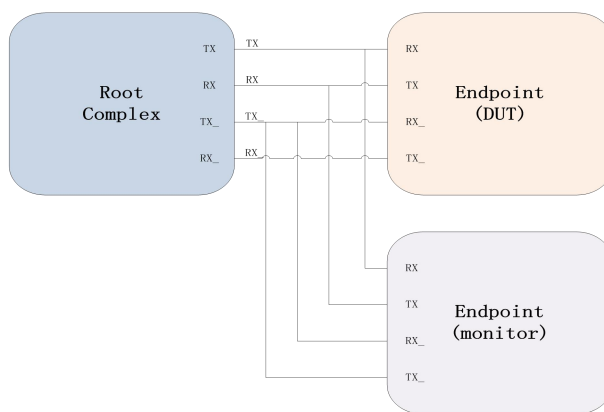


图 3.12 DUT 与 VIP model 接口连线图

在本课题中，PCIe 接口控制器分别有 PCIe1 和 PCIe2，因此存在串行查分模块 serdes1 和 serdes2，interface 接口模块的部分程序实现如图 3.13 所示。

```
interface PcieUVM_if(input clk, inputrst_n)
    parameter SETUP_TIME = 1;
    parameter HOLD_TIME = 1;
    wire [7:0] SD1_TX_TMP,NSD1_TX_TMP;
    wire [7:0] SD1_RX_TMP,NSD1_RX_TMP;
    wire [7:0] SD2_TX_TMP,NSD2_TX_TMP;
    wire [7:0] SD2_RX_TMP,NSD2_RX_TMP;

    assign SD1_TX_TMP = {SD1_TX[0:7]};
    assign {SD1_RX[0:7]} = {SD1_TX[0:7]};
    assign SD2_TX_TMP = {SD2_TX[0:7]};
    assign {SD2_RX[0:7]} = {SD2_TX[0:7]};
    assign NSD1_TX_TMP = {NSD1_TX[0:7]};
    assign {NSD1_RX[0:7]} = {NSD1_TX[0:7]};
    assign NSD2_TX_TMP = {NSD2_TX[0:7]};
    assign {NSD2_RX[0:7]} = {NSD2_TX[0:7]};
endinterface
```

图 3.13 interface 部分程序图

3.4.4 代理 agent

agent 是一个包含驱动 HDL 信号（the driver），向驱动程序（定序器）提供激励，

收集数据项以及强制执行检查和制表覆盖（监视器）所需的标准组件的设备。一个 agent 可以通过不同的实例化变量来实现不同的功能，当一个 agent 需要向接口输入数据时，这时就需要实例化一个驱动器（即 driver），当 agent 需要检测输入数据或者输出数据时，这时就需要实例化一个监视器（即 monitor）。

本课题验证平台中的 agent 为 cdnPCIEUvmAgent，并从 uvm_agent 继承。如图 3.14 所示，其中代理包含了实例化监视器（类型为 cdnPCIEUvmMonitor，引用名为 monitor），驱动器（类型为 cdnPCIEUvmDriver，引用名为 driver），激励产生器（类型为 cdnPCIEUvmSequencer，引用名为 sequencer），实例（类型为 cdnPCIEUvmInstance，引用名为 inst）和内存实例（类型为 cdnPCIEUvmMemInstance，引用名为 regInst）。只有当 UVM_ACTIVE 设置为 TRUE 时，驱动程序和序列器才会实例化。

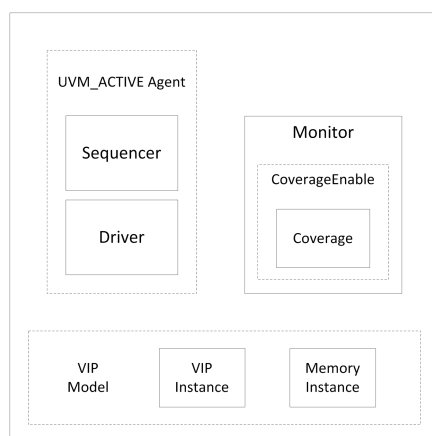


图 3.14 agent 模块

agent 的部分程序实现如图 3.15 所示。

```
class cdnPcieUvmUserAgent extends cdnPcieUvmAgent;

    cdnPcieUvmMemInstance regInst1;
    cdnPcieUvmMemInstance regInst2;

    `uvm_component_utils(cdnPcieUvmUserAgent)
    denaliPcieConfig      srcConfig;

    // new - constructor
    function new (string name = "cdnPcieUvmUserAgent", uvm_component parent = null);
        super.new(name, parent);
    endfunction : new
```

图 3.15 agent 部分程序图

3.4.5 序列 sequence

在验证的时，通过对 DUT 添加不同 testcase 来实现不同的测试激励对 DUT 进行验证，利用 sequence 产生不同的事务，对于 sequence 来说，它不属于验证平台中的一部分，但是它与 sequencer 之间有着密切的联系，在 sequencer 的帮助下，sequence 产生出的 transaction 才能最终送给 driver。UVM 中通过 objection 机制来控制验证平台关闭的，一般采用 raise_objection 和 drop_objection 来控制，但是对于 sequence 来说有一个问题：raise_objection 和 drop_objection 是 phase 函数，是 component 所特有的函数，而对于 sequence 来说它本质是一个 object，不存在 phase 的概念，所以这里需要在 uvm_sequence 中添加一个指向 phase 指针，当 sequencer 在 main_phase 中启动 default_phase 时，将 sequencer 的 main_phase 中的 phase 指针赋值到 sequence 中的指针即可，在本课题中的 sequence 中定义了一个 starting_phase 指针，只要指针不为空，就可以实现控制平台开启或者关闭。下面为部分主要程序代码如图 3.16 所示。

```

virtual task pre_start();
    super.pre_start();
`ifdef UVM_POST_VERSION_1_1
    starting_phase = get_starting_phase();
`endif
...
    if (starting_phase != null)
        starting_phase.raise_objection(this,"Sequence started...");
endtask : pre_start
...
virtual task post_body();
    super.post_body();
    if (starting_phase != null)
        starting_phase.drop_objection(this,"Sequence finished!");
endtask : post_body

```

图 3.16 sequence 部分程序图

3.4.6 激励产生器 sequencer

UVM 验证方法学的 sequencer 是一种高级激励产生组件，平台通过 sequencer 为 driver 提供数据，并且使传输数据与其他模块分离。默认情况下，sequencer 的行为类似于简单的激励发生器，并根据驱动程序的请求返回随机数据项。在本次验证环境中，激励产生器的类型为 cdnPCIEUvmUserSequencer，并从 cdnPCIEUvmSequencer 继承。其中数据项的类型为 PCIePacket。sequence 的主要作用是产生 transaction，而对于 sequencer 来说，它的作用是将 transaction 传送到 driver。如图 3.17 所示为本次验证平台中 sequencer 的部分程序代码。

```

class cdnPcieUvmUserSequencer extends cdnPcieUvmSequencer;

    denaliPcieConfig    srcConfig;
    denaliPcieConfig    dstConfig;
    cdnPcieUvmUserARcPEpEnv pEnv;

    `uvm_component_utils(cdnPcieUvmUserSequencer)

    function new (string name = "cdnPcieUvmUserSequencer" , uvm_component parent = null);
        super.new(name, parent);

    endfunction : new

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
    endfunction : build_phase

endclass

```

图 3.17 sequencer 部分程序图

3.4.7 驱动器 driver

驱动器 driver 的主要作用是收集 sequencer 发送的 transaction, 将事务包转换成事务流发送到 DUT。在 UVM 验证平台中通常 TLM (Transaction Level Modeling) 实现 component 与 transaction 级别通信, 而对于 DUT 来说, 它的端口都是 IO, 因此需要通过 driver 将接收到的事务包转化成事务流, 然后在发送给 DUT。对于本次验证平台中通过从 sequencer 获取事务包, 通过 driver 发送至实例模型。如图 3.18 所示为部分程序代码。

```

class cdnPcieUvmDriver extends uvm_driver
    cdnPcieUvmAgent pAgent;
    cdnPcieUvmInstance plnst

    `uvm_component_utils_begin(cdnPcieUvmDriver)
    `uvm_component_utils_end

    function new(string name = "cdnPcieUvmDriver", uvm_component parent = null);
        super.new(name, parent);
    endfunction : new

    virtual task run_phase(uvm_phase phase);
        super.run_phase(phase);
        mainLoop();
    endtask : run_phase
    ...
    virtual function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
    endfunction : connect_phase
endclass : cdnPcieUvmDriver

```

图 3.18 driver 部分程序图

3.4.8 监视器 monitor

对于监视器 monitor 来说,按其功能分为两种,一种主要的作用是负责监视 DUT,并且获取 DUT 的输入数据将其解码为事务包在发送至 reference model (VIP model);另一种是监视 DUT 的输出数据,并将其发送至计分板 (scoreboard)。

当将事务发送至 reference model 时,reference model 模仿 DUT 对事务包进行操作并且将结果发送至 scoreboard。当监视器监视 DUT 输出数据时,将其发送至 scoreboard,并且同 reference model 发来的信息进行 checking 以及覆盖率检查。如下图 3.19 为部分程序代码。以使能信号 coverageEnable 控制其覆盖率检测。

```

class cdnPcieUvmUserMonitor extends cdnPcieUvmMonitor;

    bit coverageEnable = 0;
    `uvm_component_utils_begin(cdnPcieUvmUserMonitor)
        `uvm_field_int(coverageEnable, UVM_ALL_ON)
    `uvm_component_utils_end

    // Coverage model
    cdnPcieUvmUserCover coverModel;

    function new(string name = "cdnPcieUvmUserMonitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction : new

    virtual function void build_phase(uvm_phase phase);
        `uvm_info("MON", "In Build Phase", UVM_LOW);
        super.build_phase(phase);

        if (coverageEnable == 1) begin
            coverModel = cdnPcieUvmUserCover::type_id::create("coverModel", this);
        end

    endfunction : build_phase

```

图 3.19 monitor 部分程序图

3.4.9 覆盖率收集器 coverage

覆盖率收集器 coverage 对从 VIP model 发送来的数据以及 monitor 检测到 DUT 的数据进行 checking，以进行覆盖率的收集。在本次验证平台设计中，用 coverage 代替了 scoreboard 的功能用于覆盖率的收集，对 PCIe 的每次事务的地址、数据、命令进行对比，比较之后正确为 pass, 错误为 fail。VIP 预先定义好协议的 coverage，数据流以及其他一切事务，如图 3.20 所示为部分程序代码。

```

function new(string name = "cdnPcieUvmUserCover", uvm_component parent = null);
    super.new(name, parent);
    covTlp = new();
    covTlp.set_inst_name({get_full_name(), ".covTlp"});
    CoverEndedCbPortImp = new("CoverEndedCbPortImp", this);
endfunction : new

virtual function void write(denaliPciePacket packet);
    if ($cast(covTlp, packet)) begin
        collectTlpCoverage();
    end
endfunction : write

virtual function void collectTlpCoverage();
    covTlp.sample();
    `uvm_info(get_type_name(), {"TLP Coverage Collected:\n"}, UVM_HIGH)
endfunction : collectTlpCoverage

```

图 3.20 coverage 部分程序图

3.4.10 VIP model

本课题中验证平台的设计与实现，其中与其他 UVM 验证平台所不一样的地方之一就是运用了 Cadence 公司提供的 PCIe VIP，在学会使用 VIP 之后，对于搭建 UVM 验证平台的效率将会很大提高，同时对以后的项目中的 VIP 使用也会有一定的经验，对于本模块 VIP model，它分为了 VIP instance 和 memory instance，VIP instance 的作用是为 VIP 模型提供了一个接口，memory instance 为提供了 VIP 模型配置和状态寄存器的接口。部分程序如图 3.21 所示：

```

class cdnPcieUvmUserMemInstance extends cdnPcieUvmMemInstance;

    uvm_event      dev_active_ev;
    uvm_event_pool pool;
    `uvm_component_utils(cdnPcieUvmUserMemInstance)
function new(string name, uvm_component parent);
    super.new(name, parent);
    dev_active_ev = new("Device_Active");
    dev_active_ev.reset();
endfunction : new

virtual task run();
    super.run();
    void'(this.setCallback(DENALI_CB_Write));
endtask : run

```

图 3.21 memory instance 部分程序图

3.5 本章小结

本章主要描述了基于 UVM 及 VIP 技术的验证平台设计,首先讲述了 VIP 组件的生成过程, VIP 组件的内部结构以及 PCIe VIP 内部的数据流, 其次详细描述了验证策略的制定, 包括了验证设计对象的描述、验证计划的制定, 验证功能点的提取, 以及验证环境的构建; 应用 UVM 验证方法学以及 VIP 技术设计了 PCIe 验证环境的架构, 并且对验证平台中各个验证组件的实现做了详细的阐述, 通过对实际项目中的验证平台搭建, 对 UVM 验证方法学有了更深的认识, 并且对 Cadence 公司的 VIP 有了更深的理解, 为以后的学习工作打下了良好的基础。

第四章 PCIe 验证平台验证实施以及覆盖率分析

在上一章中已经详细的说明了本次课题中基于 UVM 及 VIP 技术的验证平台设计, 以及介绍了各个组件在验证平台中的实现。在这一章主要说明初始化环境的配置, 以及整个验证的实现。

4.1 验证环境配置

4.1.1 X 型微处理器的上电配置

X 型微处理器的 PCIe 控制器的工作模式需要在上电的时候对管脚进行下拉来配置, 主要功能如下表所示:

表 4.1 PCIe 的端口配置

功能信号	复位配置	值	含义
TSEC2_TXD[0:3] Default (1111)	cfg_rom_loc[0:3] 用于选择启动模式	0000	PCIe 1
		0001	PCIe 2
		0010	Serial SRIO
		0100	DDR Memory Controller 1
		0101	DDR Memory Controller 2
		0110–1100	Reserved
		1101	Local bus GPCM—8-bit ROM
		1110	Local bus GPCM—16-bit ROM
TSEC1_TXD[0] Default (1)	cfg_alt_boot_vec 用于选择启动地址转换	0	PCIe1 outbound ATMU window 3 使能, 基地址被设置为 0x0_FFF0_xxxx 转换后的地址被设置为 0x0_FFFF_xxxx.
		1	取指从默认的 boot ROM (default)
TSEC4_TXD[0:3] Default (1111)	cfg_io_ports[0:3] 选择 serdes 工作模式	0000	SerDes1 与 SerDes2 关闭, 不需要参考时钟;
		0001	保留

表 4.1 PCIe 的端口配置 (续)

功能信号	复位配置	值	含义
TSEC4_TXD[0:3] Default (1111)	cfg_io_ports[0:3] 选择 serdes 工作模式	0010	SerDes1: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。 SerDes2: disabled
		0011	SerDes1: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。 SerDes2: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。(与 1111 相同)
		0100	Reserved
		0101	SerDes1: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。 SerDes2: x4 SRIO, 3.125 Gbps; 参考时钟: 125 MHz。
		0110	SerDes1: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。 SerDes2: x4 SRIO, 2.5 Gbps; 参考时钟: 100 MHz。
		0111	SerDes1: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。 SerDes2: x4 SRIO, 1.25 Gbps; 参考时钟: 100 MHz。
		1000	Reserved
		1001	SerDes1: disabled SerDes2: x4 SRIO, 3.125 Gbps; 参考时钟: 125 MHz。
		1010	SerDes1: disabled SerDes2: x4 SRIO, 2.5 Gbps; 参考时钟: 100 MHz。
		1011	SerDes1: disabled SerDes2: x4 SRIO, 1.25 Gbps; 参考时钟: 100 MHz。
		1100	Reserved

表 4.1 PCIe 的端口配置 (续)

功能信号	复位配置	值	含义
TSEC2_TXD[0:3] Default (1111)	cfg_rom_loc[0:3] 用于选择启动模式	1101	Reserved
		1110	SerDes1: disabled SerDes2: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。
		1111	SerDes1: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。 SerDes2: x1/x2/x4/x8 PCIe; 参考时钟: 100 MHz。(与上面 0011 配置一样) (default)
NLWE[2:3] Default (11)	cfg_host_agt[0:1] 选择 PICE 工作模式	00	PCIe1/PCIe2 都工作在 EP 模式
		01	PCIe1 工作在 RC 模式 PCIe2 工作在 EP 模式
		10	PCIe1 工作在 EP 模式 PCIe2 工作在 RC 模式
		11	PCIe1/PCIe2 都工作在 RC 模式

表 4.1 所示, PCIe 的端口配置需要配置 PCIe 的工作模式, 比如通 TSEC2_TXD[0:3] 选择待测设计是 PCIe1 控制器还是 PCIe2 控制器, 通过 NLWE[2:3] 的配置选择当前控制器所处的工作模式。

4.1.2 地址空间配置

1. X 处理器的地址 IO 划分

X 处理器的地址 IO 划分如图 4.1 所示:

- (1) 0x0_A000_0000-0x0_AFFF_FFFF 256Mbytes PCIe1
- (2) 0x0_B000_0000-0x0_BFFF_FFFF 256Mbytes PCIe2
- (3) 0x0_C000_0000-0x0_CFFF_FFFF 256Mbytes SRIO
- (4) 0x0_D000_0000-0x0_DFFF_FFFF 256Mbytes Local bus DSP

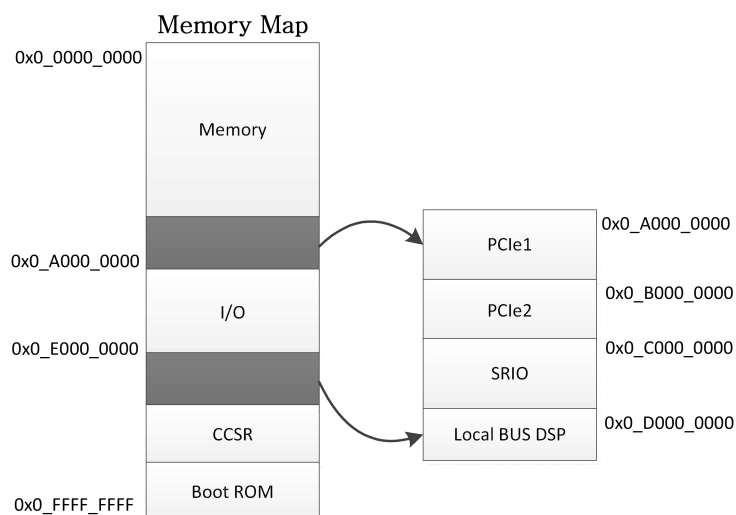


图 4.1 X 处理器 IO 地址划分

2.地址转换单元 ATMU (Address Translation and Mapping Units ATMU)

在该微处理器中,为使地址映射更灵活,定义了外部接口(SRIO,PCIe)的地址映射的地址转换与映射单元(ATMU)。

ATMU 的执行以下以功能:

- 转换本地的 36 位地址到外部地址空间 (PCIe,SRIO)。
- 转换外部地址到本地的 36 位地址空间。
- 设置转换属性。
- 映射一个本地地址到一个目标接口。
- 重新映射一个实际地址范围从 core1 到一个有区别于 core0 的物理地址范围。

Outbound 地址转换窗口必须基于选择大小域的边界对齐。Outbound 窗口未设置状态下使用 outbound ATMU window 0。交叉的 outbound 窗口是不支持的,可能产生未定义的行为。对于 RC 模式,所有的 outbound 报告事务 ATMU 必须命中在 PCIe type1 头中定义的 memory base/limit range 或者预取的 memory base/limit range。而对于 EP 模式下没有这个要求。

对于 EP 模式下的 inbound 而言,所有的基址寄存器(base address registers BARs)都是在 type0 的配置头空间中通过 PEX_CONFIG_ADDR/PEX_CONFIG_DATA 机制去进行配置的,所有的主机软件必须安排这些基址寄存器在 type0 循环中。有四个 inbound 基址寄存器。

(1) 默认的 inbound 窗口 BAR0,配置地址是 0x10 (32-bit)。它默认了 1-Mbyte 的窗口空间用于 inbound 中存储映射寄存器的配置。

(2) inbound 窗口 BAR1,配置地址是 0x14 (32-bit)。

(3) inbound 窗口 BAR2,配置地址是 0x18-0x1c (64-bit)。

(4) inbound 窗口 BAR3,配置地址是 0x20-0x24 (64-bit)。

对于 RC 模式,所有的 inbound 报告事务 ATMU 必须命中在 PCIe type1 头中定义,由对应的寄存器去配置。

3.RC 模式下的配置

在 RC 模式下, PEXIWBAR[1-3]寄存器存在于 Type1 头的外面。而 PEXIWBAR0 是唯一存在于 Type1 头的里面 (0x10)。假如事务命中了任何一个窗口,事务将会被执行并且发送给 memory,假如没有任何一个 BAR 寄存器被命中,将会向上发送一个非报告事务的完成包。所有的没有 BAR 命中的报告事务将会被忽略。

配置 RC 模式:

(1) 配置窗口寄存器 LAW,把存储器空间分配给 PCIe;对于 LAW 窗口,它分为 LAWn base address registers, 和 LAWn attributes address registers,具体的配置根据寄存器的字段进行配置,寄存器的功能描述如下表所示。

表 4.2 LAWn base address registers

位数	名字	功能描述
0--7	---	保留位
8--31	Base_addr	识别 LAWn 最重要的 24bits 地址,对于指定的地址一定要和窗口对应的大小对齐即: LAWArn[SIZE]

表 4.3 LAWn attributes registers

位数	名字	功能描述
0	EN	0: 表示 LAW 窗口被禁用 1: 表示 LAW 窗口被使能
1--6	---	保留位
7--11	TRGT	当传输事务地址命中到该窗口中时,识别该窗口对应的交互界面, TRGT 对应的译码见表 4.1 选择控制器
12--25	---	保留位
26--31	SIZE	从起始地址开始识别窗口大小,对应的窗口大小为: $2^{(size+1)}$ Byte: 000000---001010: Reserved; 001011:4Kbyte; 0011011:8Kbytes; ... $2^{(size+1)}$ bytes; 100011---111111:Reserved;

(2) 配置 Outbound Window,将分配给 PCIe 的 LAW 空间映射到外部的 PCIe (EP,SWITCH,PCIe Bridge) 系统空间,并设置好相应空间的访问属性(配置,存储

器访问，IO 访问），注意每块空间都是相互独立的，因此，可以每个 PCIe 使用 4 个窗口：

一个窗口用于配置访问，一个窗口用于存储器访问，一个窗口用于 IO 访问，另一个窗口用于 message write。这样只需要事先定义好映射空间，再等待链路初始化完成后就可以发送序列了；对于 Outbound Window 也有类似于 LAW 相关寄存器，由于篇幅原因这里就不进行详细的介绍。

（3）配置 Inbound Window，RC 模式下配置 Inbound 寄存器，Window0 是系统默认的空间映射转换，PEXITAR0 并不在存储器映射的空间中，固定 1M 的转换空间映射到 CCSRBAR 空间，系统应用时只能配置 Window1-3。配置 Inbound 属性寄存 PEXIWAR 来配置转换属性。同样 Inbound Window 也有类似于 LAW 相关寄存器，由于篇幅原因这里就不进行详细的介绍。

4. EP 模式下的配置

由第三章中 PCIe 事务表可知，EP 模式下仅支持 Mrd 与 Mwr，作为 EP 不能像作为主机一样发起配置请求，同时也不能发起 IO 访问事务。

配置 EP 模式：

（1）配置窗口寄存器 LAW，把存储器空间分配给 PCIe；

（2）配置 Outbound Window，将分配给 PCIe 的 LAW 空间映射到外部的 RC 所在系统的 PCIe 系统空间，并预先设置访问属性为 memory read or write；

（3）配置 Inbound 属性，与 RC 模式不同的是，EP 模式下使用配置头空间的 BAR0-BAR3 来配置基地址，访问方式用 PEX_CONFIG_DATA 与 PEX_CONFIG_ADDR。配置 Inbound 属性寄存 PEXIWAR 来配置转换属性。

4.2 验证流程

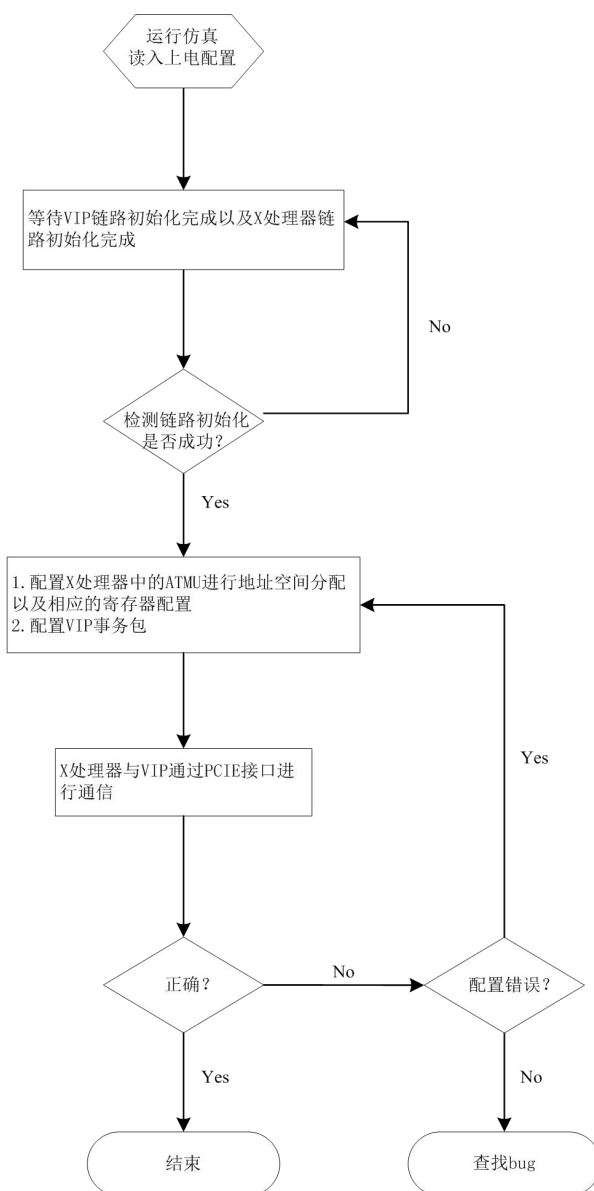


图 4.2 所示为验证流程图，首先对整个环境进行上电配置，具体的配置根据所要验证的待测设计来定，如表 4.1 所示。其次要对 VIP 的初始化检测完成和 X 处理器的链路初始化完成进行检测，链路初始化的完成是 VIP 与 PCIe 准备好交互的标志。

对于本次验证平台中 PCIe VIP 的链路初始化的完成是通过内建函数 `PCIELinkInitializedCbEvent` 捕捉到，在链路初始化完成前，验证平台保持等待状态，直到链路初始化完成。部分代码如图 4.3 所示：

```

virtual task body();
    //cast user driver to default driver
    $cast (mydrv, p_sequencer.activeMasterSeqr.pAgent.driver);
    if (starting_phase!=null)
        Starting_phase.raise_objection(this,"user_seq not finished");

    `uvm_info(get_type_name(),"Virtual sequence readAfterWriteSeq started"UVM_MEDIUM);

    //wait for the Link of PCIe initialization to finish
    p_sequence.activeMasterSeqr.pAgent.monitor.PCIELinkinitializedCbEvent.wait_trigger();
    $display("---Link innitilization finished---")

```

图 4.3 VIP 初始化部分程序图

对于 CPU 中 PCIe 的初始化完成是通过编写一个循环去检测链路状态是否到达 L0 状态决定是否链路初始化完成，当链路状态到达 L0 状态，说明链路初始化完成，否则继续循环，直到检测到链路初始化完成，CPU 的代码才会继续执行。部分代码如图 4.4 所示：

```

#####
#Loop Detect DATA from PEX_LTSSM_STAT by PEX_CONFIG_ADDR/PEX_CONFIG_DATA
#####
##### define PEX_CONFIG_ADDR=r10 #####
    ori    r10,r1,PEX_CONFIG_ADDR
##### define PEX_CONFIG_DATA=r11 #####
    ori    r11,r1,PEX_CONFIG_DATA

##### config addr:0x404 #####
    addi    r9,r0,0x0001 #COUNTER LOOP
    addi    r7,r0,0x0016 #L0 STATE
    addis   r22,r0,0x8400
    ori     r22,r22,0x0004
##### config PEX_CONFIG_ADDR #####
##### loop to detect PEX_LTSSM_STAT if into L0 then skip #####
loopreadlssd:
    addi    r9,r9,0x0001
    stw     r22,0(r10)
    lwbrx   r8,r0,r11          # read back to compare L0
    cmp     2,0,r8,r7          # compare expected with actual
    bc      4,10,loopreadlssd  # if equal skip to start test TLP

```

图 4.4 DUT 初始化部分程序图

当链路初始化完成后，通过配置相应的寄存器使 CPU 的 PCIe 接口与 VIP 进行通信。通过分析它们之间的通信结果，来确认功能的正确性或是发现设计中存在的 bug。这也是本次设计的目的，验证 PCIe 接口的正确性。

4.3 验证实施

验证的实施过程，是通过编写激励来完成验证，对于激励的编写，我们采用定向

激励测试验证边界情况以及特殊功能的情况，而对于其他情况则采用随机测试激励的编写进行验证。下面通过举例具体说明。

4.3.1 事务层验证

PCIe 接口总线的事务层的功能验证主要包括事务包的发送，接收以及流量控制三部分，由于事务包的分类模式较多，以配置读写为例进行说明，对于其他的功能验证，由于篇幅的影响，这里就不在详细说明。

以验证 PCIe RC 模式下的存储器读写为例进行说明。验证思路：通过汇编把 PCIe 配置成 RC 控制器，并且进行 ATMU 的配置，进行开窗以及地址分配配置，将 VIP 配置成 EP 模式，相当于 DUT 做主，VIP 做从，通过内核发送的写事物包、然后再针对写事物的地址发送读请求包，通过读回的数据和写入的数据进行对比，从而实现 PCIe 作为主机时发送写事物包、读请求包，以及响应包的功能验证。简化验证平台如图 4.5 所示：

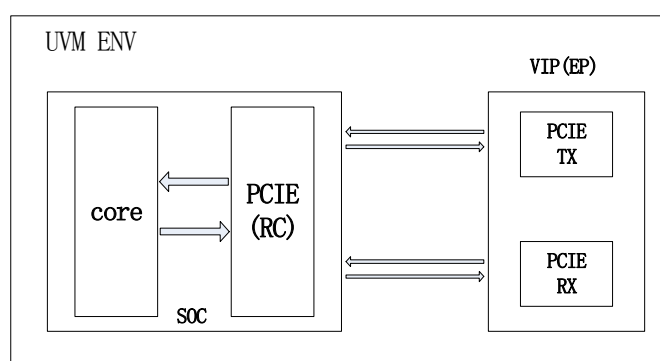


图 4.5 验证平台

如上节的验证流程所示，通过上电配置，以及 ATMU 配置，通过汇编语言经过内核对 PCIe 接口进行相关的配置，具体相关配置如下所示：

(1) 通过 LWE/LBS[2:3]信号的配置确定 PCIe 控制器的主从模式。（本例是将 PCIe 配置成 RC 模式）如图 4.6 所示：

```

////00=>1,2ep 01=>1rc2ep 10=>1ep2rc 11=>1,2rc
//*****
NLWE[2].      0      0      pull
NLWE[3].      1      0      pull

```

图 4.6 PCIe 主从模式配置图

(2) 配置 ATMU，配置事务访问方式以及地址分配。根据上节所述进行 RC 模式下的配置。详细的配置流程不一一阐述，可以通过脚本语言的编写生成日志文件，文件可以显示出验证的事务包配置，如下所示。

配置写以及写完成信号结果如图 4.7 所示:

```

95 ##### TLP COUNTER:6 #####
96 121494600.0 ps SD1_TX_TLP_Count: 3 {STP 00 03 44 00 00 01 00 00 0f 01 00 00 10 00 00 a0 14 0c 7
D}
97 Sequence Number:0003
98 DATA LCRC :140c7db6
99 TLP type: ConfigType0WriteRequest(CfgWr0)3DW_wdata
100 TC: 000
101 Warning:TD=0 NO Digest ECRC
102 Attr_order=0 PCI Strongly Ordered Model
103 Attr_Snoop=0 Hardware enforced cache coherency expected
104 TLP DATA Length:0000000001 ----- max 2^10=1024 DW(4KB) only use 6 bit 256 Byte
105 Header and DATA:00 00 00 0f 01 00 00 10 00 00 00 a0
106 Header Decode:
107 Requester ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
108 Tag: 00
109 LastDW_Be and FirstDW_Be: 0f
110 Completer ID: 0100 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
111 ExtRegNum: 00 (only [4:7])
112 RegNum: 00010000(only [0:5])
113 DATA:00 00 00 a0
114 ##### TLP COUNTER:7 #####
115 121714600.0 ps SD1_RX_TLP_Count: 2 {STP 00 02 0a 00 00 00 01 00 00 04 00 00 00 00 b3 82 fa 62 END}
116 Sequence Number:0002
117 DATA LCRC :b382fa62
118 TLP type: Completion(Cpl)3DW_nodata
119 TC: 000
120 Warning:TD=0 NO Digest ECRC
121 Attr_order=0 PCI Strongly Ordered Model
122 Attr_Snoop=0 Hardware enforced cache coherency expected
123 TLP DATA Length:0000000000 ----- max 2^10=1024 DW(4KB) only use 6 bit 256 Byte
124 Header and DATA:01 00 00 04 00 00 00 00
125 Header Decode:
126 Completer ID: 0100 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
127 CS:SuccessfulCompletion(SC)
128 ByteCountModified: 0
129 ByteCount 000000000000
130 Requester ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
131 Tag: 00
132 Lower_Address: 00(only [1:7])
133 DATA:

```

图 4.7 仿真结果图 (1)

对于事务包来说,其中主要的信息包含在包头里面,从上面的 log 文件中可以看出,包含了包的所有信息,比如包的序列号,访问方式,包的大小等,如上所示的 log 文件中,事务类型是配置写事务,事务包的大小是 1DW,包括 LCRC 校验码等,此时写的数据时 0x0000_00a0。

配置读以及读完成信号如图 4.8 所示:

```

134 ##### TLP COUNTER:8 #####
135 121962600.0 ps SD1_TX_TLP_Count: 4 {STP 00 04 04 00 00 01 00 00 0f 01 00 00 10 bd dc 24 22 END}
136 Sequence Number:0004
137 DATA LCRC :bddc2422
138 TLP type: ConfigType0ReadRequest(CfgRd0)3DW_nodata
139 TC: 000
140 Warning:TD=0 NO Digest ECRC
141 Attr_order=0 PCI Strongly Ordered Model
142 Attr_Snoop=0 Hardware enforced cache coherency expected
143 TLP DATA Length:0000000001 ----- max 2^10=1024 DW(4KB) only use 6 bit 256 Byte
144 Header and DATA:00 00 00 0f 01 00 00 10
145 Header Decode:
146 Requester ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
147 Tag: 00
148 LastDW_Be and FirstDW_Be: 0f
149 Completer ID: 0100 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
150 ExtRegNum: 00 (only [4:7])
151 RegNum: 00010000(only [0:5])
152 DATA:
153 ##### TLP COUNTER:9 #####
154 122166600.0 ps SD1_RX_TLP_Count: 3 {STP 00 03 4a 00 00 01 01 00 00 04 00 00 00 00 00 a0 11 bf 45 2.
D}
155 Sequence Number:0003
156 DATA LCRC :11bf4523
157 TLP type: CompletionW/Data(CplD)3DW_wdata
158 TC: 000
159 Warning:TD=0 NO Digest ECRC
160 Attr_order=0 PCI Strongly Ordered Model
161 Attr_Snoop=0 Hardware enforced cache coherency expected
162 TLP DATA Length:0000000001 ----- max 2^10=1024 DW(4KB) only use 6 bit 256 Byte
163 Header and DATA:01 00 00 04 00 00 00 00 00 00 00 a0
164 Header Decode:
165 Completer ID: 0100 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
166 CS:SuccessfulCompletion(SC)
167 ByteCountModified: 0
168 ByteCount 000000000000
169 Requester ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
170 Tag: 00
171 Lower_Address: 00(only [1:7])
172 DATA:00 00 00 a0

```

图 4.8 仿真结果图 (2)

如上图 4.7 和图 4.8 所示为配置写以及写完成信号和配置读以及读完成信号的数据包所有信息，从图中可以看出读回数据时跟写数据是一致的，验证功能正确。

4.3.2 数据链路层验证

PCIe 接口总线的数据链路层主要作用是链路的管理和保证数据的完整性，即 LCRC 校验以确保两台设备之间 TLP 的可靠正确传输。其中数据链路层验证功能主要包括了数据链路控制管理状态机，流控制初始化协议，以及数据链路层数据完成性管理中的重试请求与响应等。

由于验证功能点较多，这里通过对数据链路层数据完成性管理中的重试请求与响应功能进行测试用例介绍。验证思路：在正常模式下，通过 VIP 做主，DUT 做从，先跑完一组正确的波形。具体的配置跟上一节相似，这里就不详细阐述，然后再根据波形中出现的某一事务包，通过 VIP 制造一个畸形包发送给 DUT。当此事务传送到接收端时将丢弃此包，并产生一个重试请求的 DLLP 包。

仿真结果如图 4.9 所示：

```

139 ##### TLP COUNTER:9 #####
140 129020600.0 ps SD2_RX_TLP_Count: 1 {STP 00 01 4a 00 00 01 00 00 00 04 00 00 00 74 55 55 00 00 ff ff ff ff END}
141 Sequence Number:0001
142 DATA LCRC:ffffff
143 TLP type: CompletionW/Data(CpLD)3DW_wdata
144 TC: 000
145 Warning:TD=0 NO Digest ECRC
146 Attr_order=0 PCI Strongly Ordered Model
147 Attr_Snoop=0 Hardware enforced cache coherency expected
148 TLP DATA Length:0000000001 ----- max 2^10=1024 DW(4KB) only use 6 bit 256 Byte
149 Header and DATA:00 00 00 04 00 00 00 74 55 55 00 00
150 Header Decode:
151 Completer ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
152 CS:SuccessfulCompletion(SC)
153 ByteCountModified: 0
154 ByteCount 000000000000
155 Requester ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
156 Tag: 00
157 Lower_Address: 74(only [1:7])
158 DATA:55 55 00 00
159 ##### TLP COUNTER:10 #####
160 129296600.0 ps SD2_RX_TLP_Count: 2 {STP 00 01 4a 00 00 01 00 00 00 04 00 00 00 74 55 55 00 00 95 95 fc 20 END}
161 Sequence Number:0001
162 DATA LCRC:9595fc20
163 TLP type: CompletionW/Data(CpLD)3DW_wdata
164 TC: 000
165 Warning:TD=0 NO Digest ECRC
166 Attr_order=0 PCI Strongly Ordered Model
167 Attr_Snoop=0 Hardware enforced cache coherency expected
168 TLP DATA Length:0000000001 ----- max 2^10=1024 DW(4KB) only use 6 bit 256 Byte
169 Header and DATA:00 00 00 04 00 00 00 74 55 55 00 00
170 Header Decode:
171 Completer ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
172 CS:SuccessfulCompletion(SC)
173 ByteCountModified: 0
174 ByteCount 000000000000
175 Requester ID: 0000 ([0:7] BusID,[8:12] DeviceID,[13:15] FuncID)
176 Tag: 00
177 Lower_Address: 74(only [1:7])
178 DATA:55 55 00 00
179 ##### TLP COUNTER:11 #####

```

图 4.9 仿真结果图 (3)

如图 4.9 所示,当 VIP 发送一个畸形包给 DUT 时,这时 DUT 会出现 LCRC 错误,并且会重发该完成事务包。

4.3.3 物理层验证

PCIe 接口总线的物理层的主要功能是连接数据链路层和链路,决定了 PCIe 接口总线的物理特性,其中包括点对点串行传输正确性、可配置带宽和热插拔功能的实现等。

跟上述事务层以及数据链路层一样,由于验证功能点较多,这里只是举一个测试用例进行说明,在物理层,对于链路错误恢复功能进行验证,即 Link Error Recovery,验证思路:在 X8 模式下,在链路初始化的过程中,关闭 DUT 的 7 通道,通过 VIP monitor 检测,验证链路初始化是否可以正确完成。

仿真结果如图 4.10 所示:

链路完成结果如图 4.11 所示:

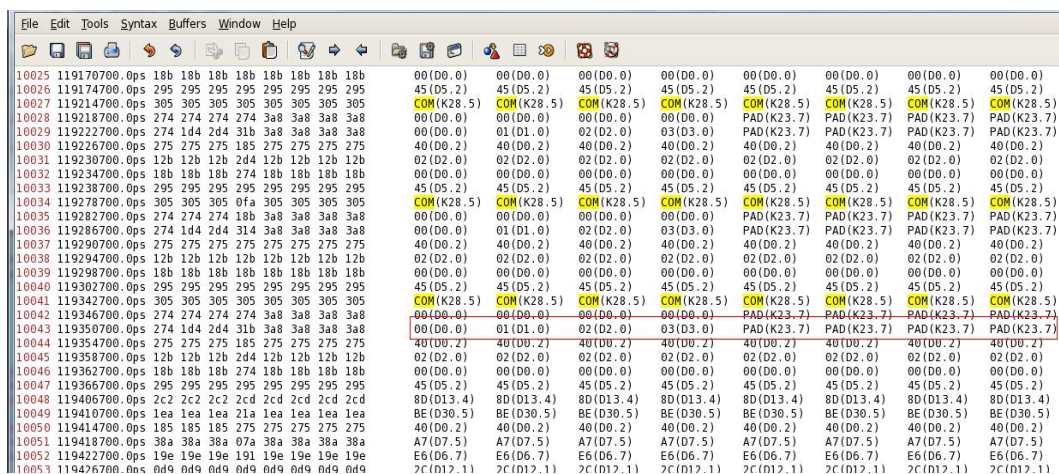


图 4.11 仿真结果图 (5)

通过上述仿真波形以及 log 文件可以看出，在 X8 链路初始化时，突然关闭 RC 的第 7 通道，链路能够以 X4 方式正常链路初始化完成，并且传递正确事务。

4.4 覆盖率相关报告

本节的主要内容是描述覆盖率测试报告，其中包括了代码覆盖率和功能覆盖率。衡量验证进展的最简单方式是使用代码覆盖率，它主要行覆盖率、条件覆盖率、翻转覆盖率以及有限状态机覆盖率^[35]，对于代码覆盖率的统计是自动化的，当运行完所有测试，代码覆盖率工具便会创建相应的数据库。代码覆盖率只是评估的设计结构是否覆盖完整，而不能体现设计的意图，也就是说不能体现设计的功能是否覆盖完全，这时候就需要对设计功能的完整性进行功能覆盖率统计，功能覆盖率它代表的是待测设计已经验证通过的功能与所有验证功能的百分比，它的重要性是与设计意图相关的，因此对于功能覆盖率来说，它不是一个自动化过程，是通过在验证之前验证人员列出的验证功能点，在仿真时创建覆盖率模型，通过模型写出验证代码，进行功能覆盖率收集^[36]。

如图 4.12 所示为 PCIe 作为 EP 模式下的接口模块的代码覆盖率。

Module Instance : PcieUVM_top.PcieUVM_if.pcie_slave_interface_instance

Instance :

SCORE	LINE	COND	TOGGLE	FSM
96.92	100.00	100.00	90.75	

Instance's subtree :

SCORE	LINE	COND	TOGGLE	FSM
96.92	100.00	100.00	90.75	

Module :

SCORE	LINE	COND	TOGGLE	FSM	NAME
96.92	100.00	100.00	90.75		pcie_slave_interface

Parent :

SCORE	LINE	COND	TOGGLE	FSM	NAME
92.84			92.84		PcieUVM_if

图 4.12 代码覆盖率

如图 4.13 所示为总的功能覆盖率图。

Total Groups coverTlperage Summary

SCORE	INST SCORE	WEIGHT
100.00	100.00	1

Total groups in report: 6

SCORE	INSTANCES	WEIGHT	GOAL	NAME
100.00	100.00	1	100	Candence_VIP_PACKET::dw_vip_PcieUVM_monitor_def_coverTlp_callbacks::MEM_MAP_REGS
100.00	100.00	1	100	Candence_VIP_PACKET::dw_vip_PcieUVM_monitor_def_coverTlp_callbacks::ERROR_MSG
100.00	100.00	1	100	Candence_VIP_PACKET::dw_vip_PcieUVM_monitor_def_coverTlp_callbacks::Tlp_Mrd
100.00	100.00	1	100	Candence_VIP_PACKET::dw_vip_PcieUVM_monitor_def_coverTlp_callbacks::Tlp_MWr
100.00	100.00	1	100	Candence_VIP_PACKET::dw_vip_PcieUVM_monitor_def_coverTlp_callbacks::Tlp_Cpl
100.00	100.00	1	100	Candence_VIP_PACKET::dw_vip_PcieUVM_monitor_def_coverTlp_callbacks::Tlp_CplD

图 4.13 功能覆盖率

4.5 验证平台的重用性研究

目前业界对于验证所关注的重点就是验证平台的重用性^[37]。而对于 UVM 验证平台来说，它相比较于其他验证平台的一大优势，就是在于验证平台的可重用性。重用主要分为横向重用和纵向重用，横向重用主要指的是验证组件在不同项目中的重用，而对于纵向重用的是在同一个项目的验证的不同时期对早期资源的重复利用^[38]。

由上面章节可知本次课题中的待测设计是 X 型微处理器的外设 PCIe 接口，课题中需要验证两个功能相同的 PCIe 控制器，即 PCIe1 和 PCIe2，由于两者功能相同，只是地址空间分配和接口信号不同，因此在验证平台的设计中，可以使用纵向的可重用，即在验证过程中只需要搭建两个 In_agent，如图 3.10 所示，一个用于验证 PCIe 控制器为 RC 的模式，一个用于 PCIe 控制器为 EP 的模式，而对于 PCIe1 和 PCIe2，可以重用 In_agent，只需要变动 sequence_lib 里面的 sequence 队列，同时在实例化接口时，实例化对应的 PCIe 控制器接口即可。在验证的实施时，分为主动模式跟被动模式，主动模式由 VIP 进行包的生成通信，被动模式由 CPU 发送命令进行包的生成通信，两种模式重用 agent 组件，只需改变 cfg 类的配置，将 VIP 组件配置成相应的配置模式即可。

4.6 本章小结

本章主要描述了整个 PCIe 验证平台的实施，通过验证平台的初始化配置，分别通过取 PCIe 接口中的事务层、数据链路层以及物理层各层功能中的一个测试用例，对验证的实施进行了描述，同时也列出了验证的相关仿真结果，其次通过覆盖率的描述，以及覆盖率的收集结果说明对于 PCIe 接口的验证达到了验证之前的验证目标，完成了验证任务，最后对本课题中验证平台的重用进行了说明。通过本次验证的实施，让验证工程师不仅对 PCIe 接口的功能有了深刻的理解，并且对 UVM 验证平台以及 VIP 组件的应用也有了一定的长足的进步。

第五章 总结与展望

5.1 工作总结

本文是针对基于 PowerPC 架构下的 X 型微处理器的外设 PCIe 接口进行功能验证, 作为 X 处理器的外设接口, 本次作为待测设计的 PCIe 接口在遵循 PCIe 接口协议的同时需要通过内核完成事务的处理, 因此在验证过程中需要使用汇编语言通过内核对 PCIe 接口功能进行相应的配置。本次验证平台使用 UVM 验证方法学以及结合 Cadence 公司 VIP 组件进行设计。通过 UVM 及 VIP 技术的使用, 来改善验证的效率, 减少验证时间。主要的研究内容和研究成果归纳如下:

(1) 对 PCIe 接口总线协议和 UVM 验证方法学的关键技术进行了分析研究, 掌握了 PCIe 接口协议的主要内容以及关键技术, 其中主要包括 PCIe 的数据包各个字段的含义, 以及明确了 UVM 验证方法学的结构特点和分层设计搭建验证平台的方法。同时对 Cadence 公司的 VIP 组件进行了分析研究, 可以熟练的对 VIP 组件进行生成和应用。

(2) 针对本课题中的 PCIe 接口进行了基于 UVM 及 VIP 组件技术的验证平台搭建, 从验证对象的分析, 验证计划的制定以及验证功能点的列举, 最后再到验证环境的实现耗费了大量时间。不仅对整验证平台个验证策略进行了详细的说明, 而且详细阐述了验证平台中各个组件的功能以及部分代码实现。

(3) 为了证明本次验证平台的正确性, 通过编写激励将待测设计进行了验证, 通过实际验证过程中的激励举例, 将验证产生的仿真波形以及 log 文件进行了结果说明; 其次通过覆盖率的收集, 使代码覆盖率达到 90% 以上, 功能覆盖率达到 100%, 完成了待测设计的功能验证; 最后对本课题中验证平台的可重用进行了分析说明。

5.2 后续展望

本文对作为 X 型微处理器外设的 PCIe 接口进行了验证, 花了大量的时间进行了验证平台的搭建以及测试用例的编写, 最终搭建好了验证平台, 并且对待测设计的正确性进行了验证。然而对于作为第三代总线协议的 PCIe 接口协议, 其协议复杂, 并且更新较快, 同样 UVM 验证方法学也在不断地更新, 随着各大公司 VIP 产品的成熟, 对于 PCIe 这么一个应用广泛的协议仍有很多验证方法上的改进的地方。

(1) 要更加深入的研究 UVM 验证方法学, 在应对复杂设计的验证过程中, 可以搭建出更易操控, 重用性更好的验证平台, 其次要提高自动化水平, 掌握 Perl 语言的使用, 使工作更有效率。

(2) 对现有的 Cadence 公司的 VIP 产品进行深入分析研究, 掌握其功能, 它的产品较多, 通过学习应用, 不仅可以提高验证效率, 更能开阔思维, 避免重复性劳动。为日后的学习工作提供便利。

参考文献

- [1] 骆丽, 程成.采用 UVM 的数字抽取滤波器的验证[J].北京交通大学学报.2014(2): 24-28.
- [2] 彭成. 图象处理系统中去噪模块的设计及基于 SystemVerilog 的系统级验证[D].合肥:合肥工业大学.2007.
- [3] 曹阳, 胡越黎. 基于 UVM 的存储器功能验证[J].上海大学.微电子研究与开发中心, 2015(3): 834-837.
- [4] 强俊,楚宁,周鸣争. 基于 PCI EXPRESS 总线的视音频采集卡的设计[J].《计算机技术与发展》,2007(5):79-82.
- [5] 王进. 基带芯片中 GST 模块验证方法的研究与实现[D].西安: 西安电子科技大学.2014.
- [6] 陈新. FLASH 控制器 IP 的功能验证研究[D].西安: 西安电子科技大学.2010.
- [7] 周德新, 金志威等. 基于 SystemVerilog 的 SRAM 控制器 IP 核验证[J].《电子器件》,2012(5):619-622.
- [8] 徐栋磊. VIP 及其在验证平台中的应用的研究[D].成都: 西南交通大学, 2007.
- [9] 张功萱等.《重点大学计算机专业系列教材 计算机组成原理》[M].北京:清华大学出版社.2009.9.1.
- [10] 刘洋. 基于 FPGA 的 CT 图像三维重建系统中 PCIe DMA 子系统的研究[D].哈尔滨: 哈尔滨工业大学.2010.
- [11] 周南岚. 基于 PCI Express 高速数据传输接口的技术研究[D].南京:南京理工大学.2011.
- [12] 李文举. 高速图像压缩芯片高速自动化测试技术研究[D].西安: 西安电子科技大学.2013.
- [13] 胡永鑫. PCIe3.0QoS 的电路设计[D].南京: 东南大学.2014.
- [14] 李仁杰. 基于 PCI-E3.0 的高速网卡中 DMA 的设计[D].南京: 东南大学.2014.
- [15] Sardella S D, Sullivan D. Root complex connection system: U.S. Patent 7,062,594[P]. 2006.6.13.
- [16] Brown D A. Method and apparatus for an aggregated non-transparent requester ID translation for a PCIe switch: U.S. Patent 8,917,734[P]. 2014.12.23.
- [17] 丁燕. PCIe3.0 数据链路层的电路设计[D].南京: 东南大学.2014.
- [18] 闵牡丹,刘一娜. PCIe 总线的 FPGA 设计方法[J].《计算机与现代化》,2009(8):178-182,186.
- [19] 赵会彬, 马卫平, 梁晓英. 基于 PCIe 点对点传输的 FPGA 系统[J].《计算机系统应用》.2014(4):201-204.
- [20] 牛晓军. 基于 PCI Express 总线高速数据采集系统的设计[D].重庆: 重庆大学.2013.
- [21] 王映. 螺旋锥束 CT 三维重建 FPGA 硬件加速系统中 PCIe DMA 的设计与实现[D].哈尔滨: 哈尔滨工业大学.2009.
- [22] 关凯峰. PCIe 事务层及数据链路层的实现与验证[D].西安: 西安电子科技大学.2013.

- [23] 张亮. PCIe 总线物理层的设计与验证[D].西安: 西安电子科技大学.2013.
- [24] IEEE1800-2009. SystemVerilog Unified Hardware Design,Specification,and Verification Language.New York,IEEE Computer Society[S].2011.
- [25] 雷霆. 基于 UVM 的网络数据包解析器的验证研究[D].成都: 电子科技大学.2015.
- [26] Accellera Universal Verification Methodology(UVM) 1.1User's Guide[M].2011.
- [27] 秦娟. 基于 Magellan 和 UVM 验证方法的研究及应用[D].南京: 东南大学.2013.
- [28] 李保华,张丽. 一种基于 SoC/IP 验证平台的设计实现[J]. 《环球市场信息导报》,2012(2):48-49.
- [29] 张强. UVM 实战[M]. 机械工业出版社, 2014.
- [30] 李俊. 基于 UVM 的视频图像 2D 转 3D 模块的验证[D].青岛: 中国海洋大学.2013.
- [31] Cadence. PCI Express VIP User Interface Reference[M].2016.
- [32] Cadence. PCI Express VIP User Guide[M].2015.
- [33] 刘俊池. 基于 SoC 的 PCIe3.0 控制器的实现与验证[A]. 中国计算机学会.第十八届计算机工程与工艺年会暨第四届微处理器技术论坛论文集[C].中国计算机学会:,2014:5.
- [34] 程海燕. 基于 UVM 架构的 EHCI 验证环境研究与开发[D].成都: 电子科技大学.2014.
- [35] 严小兵, 张毅. 基于 Specman 的嵌入式软件验证的设计流程[J]. 《计算机与现代化》.2010(8):182-184.
- [36] 尹飞. 基于功能覆盖率验证多核处理器中的 Cache 一致性协议[A]. 中国计算机学会.第十五届计算机工程与工艺年会暨第一届微处理器技术论坛论文集 (A 辑) [C].中国计算机学会:,2011:6.
- [37] 孟宾. 基于 UVM 验证方法学的 MCU 验证[A]. 中国计算机学会.第十七届计算机工程与工艺年会暨第三届微处理器技术论坛论文集 (上册) [C].中国计算机学会:,2013:5.
- [38] 谢峥, 王腾, 雍珊珊, 等. 一种基于 UVM 面向 RISCCPU 的可重用功能验证平台[J]. 北京大学学报 (自然科学版), 2014, 2:004.

致谢

指缝很宽，时间太瘦，时光悄悄的就从指缝间溜走了。转眼间三年的研究生阶段就要结束了，心中充满了对学校的不舍，同时又充满了想离开象牙塔出去好好闯一闯的心情。但是时光一去不复返，是到离别的时候了。回想这三年，从研一的课本学习，研二的外出实习，研三的论文撰写，在学校提供的优秀学习平台基础上，使我的整个硕士研究生生活丰富多彩，并且通过老师同学的帮助，同时使我受益匪浅。

首先要感谢我的导师宣荣喜老师，老师平易近人的魅力以及豁达的胸怀，使我们再跟老师交流的时候很轻松很舒服，可以真切的表达内心的想法，并且老师可以耐心的解答我们的疑问，不管是学习上还是生活中都给了我们很大的帮助；同时老师渊博的专业知识、对科研工作的热情、严谨的工作态度给我们留下了深刻的印象。从论文的开题，中期以及最后完稿宣老师对我们严格把关，使我们可以顺利的完成论文工作，谢谢老师。

感谢我的经理吴经理以及我的两位师傅邱钧华数字验证工程师和高新军数字验证工程师，如果课本的知识使我了解了 IC 行业，那么你们是带我进 IC 行业的领路人，两位师傅严谨的科研态度，开阔的思维给了我很大的帮助，而且对我在工作中的疑问都可以耐心解答，使我受益匪浅。

感谢实习公司的同事，你们给了我一个很好地学习科研的环境。感谢与我一起实习的李腾、张新展，在工作的过程中我们互帮互助，讨论问题，解决问题，在生活中我们可以一起玩耍游玩，有了你们工作生活不再枯燥乏味，而是充满欢乐。

最后要特别感谢我的父母，是你们将我养大，供我上学，在学习中的费用从来都不加吝啬，无条件的支持我，可以让我安心的学习，是你们的支持让我有了更大的动力，去创造美好的明天。



西安电子科技大学
XIDIAN UNIVERSITY

地址：西安市太白南路2号

邮编：710071

网址：www.xidian.edu.cn